

Research in Computational Biology book

Gepoliano Chaves

2026-07-16

Contents

1	Prerequisites	5
2	Basics of R	7
2.1	Basic Steps	7
2.2	Library ISwR	7
2.3	Plot Random Numbers	8
2.4	A Potent Calculator	8
2.5	Assignments	9
2.6	Operations after assignment of variable	9
2.7	Vectorized Arithmetic	10
2.8	Calculate the Mean of the variable weight	11
2.9	Calculate the Standard-Deviation of the variable weight	12
2.10	Calculation of the Standard Deviation	12
2.11	Standard Statistical Procedures	13
2.12	Standard Statistical Procedures: t-test	13
2.13	Plot Graphics	14
2.14	Plot Graphics Modifying the plotting character	15
2.15	Plot an expected Line for weights at BMI of 22.5	15
2.16	Vectors	16
2.17	Using other libraries	16
2.18	More Visualization	22
2.19	Data Visualization with specialized libraries	25
2.20	Create vector to highlight genome coordinates in Manhattan plot	31

2.21 Biological Association Visualization: Manhattan Plot	33
2.22 References	34
2.23 Session Info	34
3 Processamento de dados públicos de neuroblastoma	35
3.1 Origem dos dados	35
3.2 Carregando dados pre-processados de R2	35
3.3 Primeiros gráficos com dados de neuroblastoma	37
4 Análise de Sobrevivência	41
4.1 Processamento de variáveis para permitir a construção de gráficos	41
4.2 Fazendo a análise de sobrevivência	42
5 Accessing the Raça/Cor Variable in DATASUS	53
5.1 Load libraries	53
5.2 Constrir dataframe (df) com identificação racial	53
5.3 Transformar variável Raça/Cor	54
5.4 Gráfico de barras por categoria da Variável Raça/Cor	54
5.5 Mortalidade Anual Total	56
5.6 Mortalidade Anual Total por Neuroblastoma	63
6 Final Words	75

Chapter 1

Prerequisites

This is a *sample* book written in **Markdown**. You can use anything that Pandoc's Markdown supports, e.g., a math equation $a^2 + b^2 = c^2$.

The **bookdown** package can be installed from CRAN or Github:

```
install.packages("bookdown")  
# or the development version  
# devtools::install_github("rstudio/bookdown")
```

Remember each Rmd file contains one and only one chapter, and a chapter is defined by the first-level heading #.

To compile this example to PDF, you need XeLaTeX. You are recommended to install TinyTeX (which includes XeLaTeX): <https://yihui.org/tinytex/>.

Chapter 2

Basics of R

2.1 Basic Steps

- R can be accessed by clicking an icon or entering the command “R” at the system command line, also referred to as Terminal
- This produces a console window or causes R to start up as an interactive program at the current terminal window
- R works fundamentally by a question-and-answer model: enter a line with a command and press Enter
- Then the program does something relevant
- In this course, we will use RStudio and a file format called R Markdown, which allows us to code using not only R, but bash and Python, two other languages we may refer to as the course progresses

2.2 Library ISwR

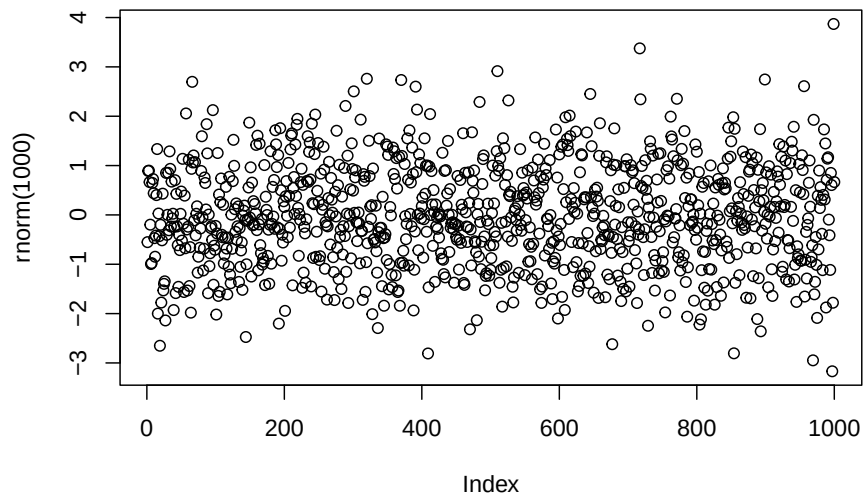
- This notebook used the book Introductory Statistics with R, by Peter Dalgaard as a reference
- For the book, R library ISwR (Introductory Statistics with R) can be freely downloaded
- All examples in the book used as reference should run provided that ISwR library is not only installed but also loaded into the current search path
- Library can be installed and loaded by typing the following command into an R chunk

```
install.packages("ISwR")  
library("ISwR")
```

2.3 Plot Random Numbers

- For a first impression of what R can do, let's try plotting a graph
- Need to insert the R chunk and use the plot function

```
plot(rnorm(1000))
```



2.4 A Potent Calculator

- R can process simple and complex arithmetic expression and produce a result for the user

```
2 + 2
```

```
## [1] 4
```


- R can also be used to do other standard calculations. Here is how to calculate e to the power of -2

```
exp(-2)
```

```
## [1] 0.1353353
```

- Other than the R chunks, these calculations can be made using the RStudio Console
- In RStudio, when using the R Markdown format (.Rmd), we can insert R, Bash (terminal) and Python chunks
- Proceed to illustrate that in the RStudio IDE

2.5 Assignments

- Assignments are made based on the necessity to store the results of calculations and use these results in downstream processing steps in an entire algorithm or pipeline
- Like other languages, R has symbolic **variables**: names that can be used to represent values
- To assign the value 2 to variable x, we can enter the following command

```
x <- 2
```

- The character <- is called the assignment operator
- There is no immediate visible result, but from now on, x has the value 2 and can be used in subsequent arithmetic operations
- We can “ask” R which value is stored in the x variable

```
x
```

```
## [1] 2
```

2.6 Operations after assignment of variable

- Below, our variable x, is used to perform other calculations

```
x
```

```
## [1] 2
```

- Addition

```
x + x
```

```
## [1] 4
```

- Multiplication

```
5*x
```

```
## [1] 10
```

- Potentiation

```
x^3
```

```
## [1] 8
```

2.7 Vectorized Arithmetic

- It is not useful to use one number at a time to run statistics
- One strength of R is that it can handle entire data vectors as single objects
- A data vector is an array of numbers and a vector variable can be constructed like this

```
weight <- c(60, 72, 57, 90, 95, 72)
```

```
## To look at the vector variable, just type its name again  
weight
```

```
## [1] 60 72 57 90 95 72
```

- You can do calculations with vectors just like ordinary numbers, so long as they have the same length

- One exception to this rule that we will see will be when we use the mean of weights of persons (represented by $\bar{x}$)
- In that case, the mean will be one single number, which will be subtracted from each sample value
- Suppose the weight vector indicates the weight of men in kilograms
- One simple formula to indicate whether a person is obese or not, is the body mass index (BMI)
- BMI is calculated by dividing the person's weight by the square of their height, in meters
- Therefore, in R, we need to have a vector with the height values to calculate the bmi vector, containing the body-mass index for the individuals indicated in the weight vector

```
height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
bmi <- weight/height^2
bmi
```

```
## [1] 19.59184 22.22222 20.93664 24.93075 31.37799 19.73630
```

2.8 Calculate the Mean of the variable weight

- The mean is calculated by the sum of the observations divided by the total number of observations

$$\bar{x} = \sum x_i / n$$

- Let's calculate the mean of the variable weight

```
sum(weight)
```

```
## [1] 446
```

```
sum(weight)/length(weight)
```

```
## [1] 74.33333
```

2.9 Calculate the Standard-Deviation of the variable weight

- Standard-deviation can be calculated with the following equation

$$SD = \sqrt{(\sum (x_i - \bar{x})^2) / (n - 1)}$$

- xbar, the mean of variable weight, can be calculated using the sum and length of variable weight

```
xbar <- sum(weight)/length(weight)
xbar
```

```
## [1] 74.33333
```

- Now we can calculate the difference of each replicate in the weight variable and the mean of the weight variable, one by one

```
weight - xbar
```

```
## [1] -14.333333 -2.333333 -17.333333 15.666667 20.666667 -2.333333
```

- Notice how R uses xbar, which has length one, to calculate the new weight - xbar data vector
- xbar is recycled and subtracted from every element in the weight variable (weight data vector)

2.10 Calculation of the Standard Deviation

- Calculate the squared deviations

```
(weight - xbar)^2
```

```
## [1] 205.444444 5.444444 300.444444 245.444444 427.111111 5.444444
```

- Calculate the sum of squared deviations

```
sum((weight - xbar)^2)
```

```
## [1] 1189.333
```

Calculate the standard deviation

```
sqrt(  
  sum(  
    (weight - xbar)^2/  
    (length(weight)-1)  
  )  
)
```

```
## [1] 15.42293
```

2.11 Standard Statistical Procedures

- It is a standard medical practice to assess whether a person is obese or not using validated scientific criteria
- As a simple procedure to show this concept, let's assume that an individual with a normal weight should have a BMI in the range 20-25
- We want to know if our data deviates from the normal range of BMI
- In R, this can be done using a statistical test called t-test
- You do not need to understand what a t-test is, just remember that it is used to evaluate the distribution of sample values compared to the normal distribution
- You can use a one-sample t-test to assess whether the six persons' BMI can be assumed to have mean 22.5 given that they come from a normal distribution
- You can do that using the function `t.test`

2.12 Standard Statistical Procedures: t-test

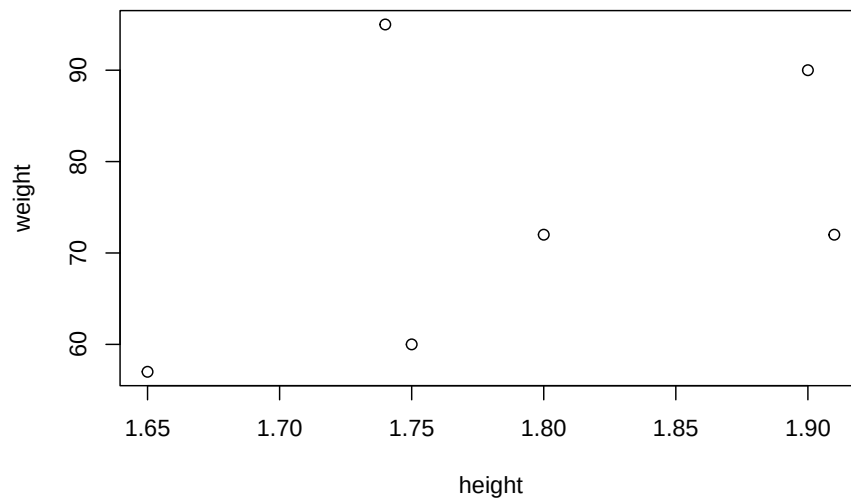
```
t.test(  
  bmi,  
  mu = 22.5  
)
```

```
##  
## One Sample t-test  
##  
## data:  bmi  
## t = 0.34488, df = 5, p-value = 0.7442  
## alternative hypothesis: true mean is not equal to 22.5  
## 95 percent confidence interval:  
##  18.41734 27.84791  
## sample estimates:  
## mean of x  
##  23.13262
```

2.13 Plot Graphics

- Let's now plot a scatterplot of the height and weight of individuals

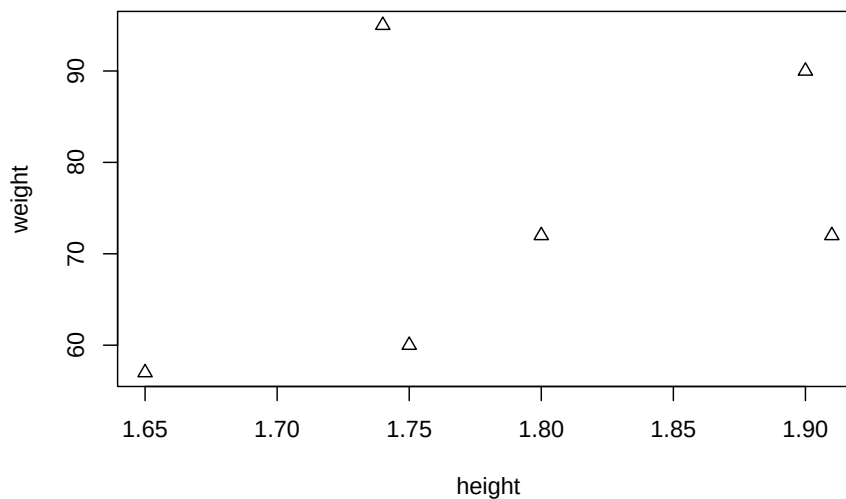
```
plot(height, weight)
```



2.14 Plot Graphics Modifying the plotting character

- You will frequently want to modify drawing of your graphs in various ways
- One way is using the parameter “plotting character”, `pch`

```
plot(height, weight, pch = 2)
```



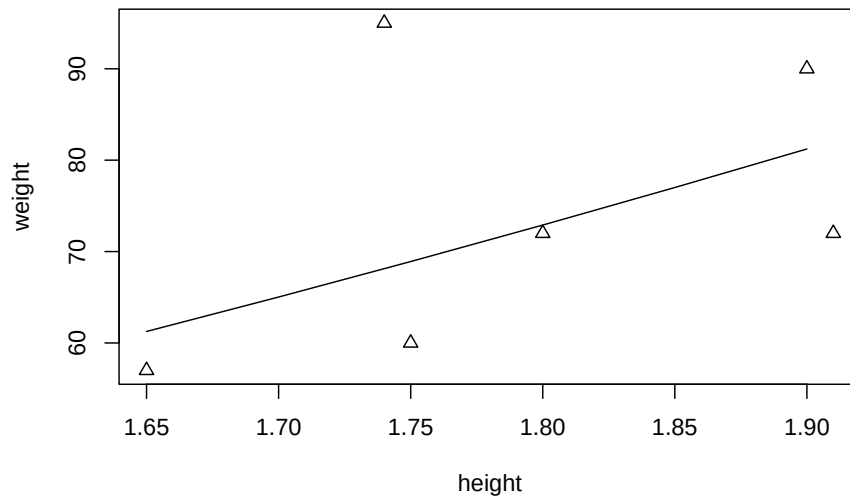
2.15 Plot an expected Line for weights at BMI of 22.5

- The term for weight at BMI of 22.5 is:

```
22.5*(height)^2
```

- This term can be used to compute the line of expected weights at a BMI of 22.5 using the `lines()` function
- As a note, we can't use the `lines()` function without first creating a plot in R.

```
plot(height, weight, pch = 2)
hh <- c(1.65, 1.70, 1.75, 1.80, 1.85, 1.90)
lines(hh, 22.5 * (hh)^2)
```



2.16 Vectors

- The weight and height vectors are called numeric vectors
- Besides numeric vectors, there are numeric and character vectors

2.17 Using other libraries

2.17.1 The iris data-frame

- Now we have a better understanding of what R can give us, let us use another library more commonly used datasets
- At times, it is possible that you will need to figure out different ways to install a library to use it


```
library(datasets)
```

- In the next chunk, we access the iris data, and look at a summary of the dataset

```
data(iris)
summary(iris)
```

```
##      Sepal.Length      Sepal.Width      Petal.Length      Petal.Width
##  Min.   :4.300      Min.   :2.000      Min.   :1.000      Min.   :0.100
##  1st Qu.:5.100      1st Qu.:2.800      1st Qu.:1.600      1st Qu.:0.300
##  Median :5.800      Median :3.000      Median :4.350      Median :1.300
##  Mean   :5.843      Mean   :3.057      Mean   :3.758      Mean   :1.199
##  3rd Qu.:6.400      3rd Qu.:3.300      3rd Qu.:5.100      3rd Qu.:1.800
##  Max.   :7.900      Max.   :4.400      Max.   :6.900      Max.   :2.500
##           Species
##  setosa      :50
##  versicolor:50
##  virginica   :50
##
##
##
```

- Another form to look at the iris data-frame is typing its name

```
iris
```

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width      Species
## 1           5.1           3.5           1.4           0.2      setosa
## 2           4.9           3.0           1.4           0.2      setosa
## 3           4.7           3.2           1.3           0.2      setosa
## 4           4.6           3.1           1.5           0.2      setosa
## 5           5.0           3.6           1.4           0.2      setosa
## 6           5.4           3.9           1.7           0.4      setosa
## 7           4.6           3.4           1.4           0.3      setosa
## 8           5.0           3.4           1.5           0.2      setosa
## 9           4.4           2.9           1.4           0.2      setosa
## 10          4.9           3.1           1.5           0.1      setosa
## 11          5.4           3.7           1.5           0.2      setosa
## 12          4.8           3.4           1.6           0.2      setosa
## 13          4.8           3.0           1.4           0.1      setosa
## 14          4.3           3.0           1.1           0.1      setosa
## 15          5.8           4.0           1.2           0.2      setosa
```

## 16	5.7	4.4	1.5	0.4	setosa
## 17	5.4	3.9	1.3	0.4	setosa
## 18	5.1	3.5	1.4	0.3	setosa
## 19	5.7	3.8	1.7	0.3	setosa
## 20	5.1	3.8	1.5	0.3	setosa
## 21	5.4	3.4	1.7	0.2	setosa
## 22	5.1	3.7	1.5	0.4	setosa
## 23	4.6	3.6	1.0	0.2	setosa
## 24	5.1	3.3	1.7	0.5	setosa
## 25	4.8	3.4	1.9	0.2	setosa
## 26	5.0	3.0	1.6	0.2	setosa
## 27	5.0	3.4	1.6	0.4	setosa
## 28	5.2	3.5	1.5	0.2	setosa
## 29	5.2	3.4	1.4	0.2	setosa
## 30	4.7	3.2	1.6	0.2	setosa
## 31	4.8	3.1	1.6	0.2	setosa
## 32	5.4	3.4	1.5	0.4	setosa
## 33	5.2	4.1	1.5	0.1	setosa
## 34	5.5	4.2	1.4	0.2	setosa
## 35	4.9	3.1	1.5	0.2	setosa
## 36	5.0	3.2	1.2	0.2	setosa
## 37	5.5	3.5	1.3	0.2	setosa
## 38	4.9	3.6	1.4	0.1	setosa
## 39	4.4	3.0	1.3	0.2	setosa
## 40	5.1	3.4	1.5	0.2	setosa
## 41	5.0	3.5	1.3	0.3	setosa
## 42	4.5	2.3	1.3	0.3	setosa
## 43	4.4	3.2	1.3	0.2	setosa
## 44	5.0	3.5	1.6	0.6	setosa
## 45	5.1	3.8	1.9	0.4	setosa
## 46	4.8	3.0	1.4	0.3	setosa
## 47	5.1	3.8	1.6	0.2	setosa
## 48	4.6	3.2	1.4	0.2	setosa
## 49	5.3	3.7	1.5	0.2	setosa
## 50	5.0	3.3	1.4	0.2	setosa
## 51	7.0	3.2	4.7	1.4	versicolor
## 52	6.4	3.2	4.5	1.5	versicolor
## 53	6.9	3.1	4.9	1.5	versicolor
## 54	5.5	2.3	4.0	1.3	versicolor
## 55	6.5	2.8	4.6	1.5	versicolor
## 56	5.7	2.8	4.5	1.3	versicolor
## 57	6.3	3.3	4.7	1.6	versicolor
## 58	4.9	2.4	3.3	1.0	versicolor
## 59	6.6	2.9	4.6	1.3	versicolor
## 60	5.2	2.7	3.9	1.4	versicolor
## 61	5.0	2.0	3.5	1.0	versicolor

## 62	5.9	3.0	4.2	1.5 versicolor
## 63	6.0	2.2	4.0	1.0 versicolor
## 64	6.1	2.9	4.7	1.4 versicolor
## 65	5.6	2.9	3.6	1.3 versicolor
## 66	6.7	3.1	4.4	1.4 versicolor
## 67	5.6	3.0	4.5	1.5 versicolor
## 68	5.8	2.7	4.1	1.0 versicolor
## 69	6.2	2.2	4.5	1.5 versicolor
## 70	5.6	2.5	3.9	1.1 versicolor
## 71	5.9	3.2	4.8	1.8 versicolor
## 72	6.1	2.8	4.0	1.3 versicolor
## 73	6.3	2.5	4.9	1.5 versicolor
## 74	6.1	2.8	4.7	1.2 versicolor
## 75	6.4	2.9	4.3	1.3 versicolor
## 76	6.6	3.0	4.4	1.4 versicolor
## 77	6.8	2.8	4.8	1.4 versicolor
## 78	6.7	3.0	5.0	1.7 versicolor
## 79	6.0	2.9	4.5	1.5 versicolor
## 80	5.7	2.6	3.5	1.0 versicolor
## 81	5.5	2.4	3.8	1.1 versicolor
## 82	5.5	2.4	3.7	1.0 versicolor
## 83	5.8	2.7	3.9	1.2 versicolor
## 84	6.0	2.7	5.1	1.6 versicolor
## 85	5.4	3.0	4.5	1.5 versicolor
## 86	6.0	3.4	4.5	1.6 versicolor
## 87	6.7	3.1	4.7	1.5 versicolor
## 88	6.3	2.3	4.4	1.3 versicolor
## 89	5.6	3.0	4.1	1.3 versicolor
## 90	5.5	2.5	4.0	1.3 versicolor
## 91	5.5	2.6	4.4	1.2 versicolor
## 92	6.1	3.0	4.6	1.4 versicolor
## 93	5.8	2.6	4.0	1.2 versicolor
## 94	5.0	2.3	3.3	1.0 versicolor
## 95	5.6	2.7	4.2	1.3 versicolor
## 96	5.7	3.0	4.2	1.2 versicolor
## 97	5.7	2.9	4.2	1.3 versicolor
## 98	6.2	2.9	4.3	1.3 versicolor
## 99	5.1	2.5	3.0	1.1 versicolor
## 100	5.7	2.8	4.1	1.3 versicolor
## 101	6.3	3.3	6.0	2.5 virginica
## 102	5.8	2.7	5.1	1.9 virginica
## 103	7.1	3.0	5.9	2.1 virginica
## 104	6.3	2.9	5.6	1.8 virginica
## 105	6.5	3.0	5.8	2.2 virginica
## 106	7.6	3.0	6.6	2.1 virginica
## 107	4.9	2.5	4.5	1.7 virginica

## 108	7.3	2.9	6.3	1.8	virginica
## 109	6.7	2.5	5.8	1.8	virginica
## 110	7.2	3.6	6.1	2.5	virginica
## 111	6.5	3.2	5.1	2.0	virginica
## 112	6.4	2.7	5.3	1.9	virginica
## 113	6.8	3.0	5.5	2.1	virginica
## 114	5.7	2.5	5.0	2.0	virginica
## 115	5.8	2.8	5.1	2.4	virginica
## 116	6.4	3.2	5.3	2.3	virginica
## 117	6.5	3.0	5.5	1.8	virginica
## 118	7.7	3.8	6.7	2.2	virginica
## 119	7.7	2.6	6.9	2.3	virginica
## 120	6.0	2.2	5.0	1.5	virginica
## 121	6.9	3.2	5.7	2.3	virginica
## 122	5.6	2.8	4.9	2.0	virginica
## 123	7.7	2.8	6.7	2.0	virginica
## 124	6.3	2.7	4.9	1.8	virginica
## 125	6.7	3.3	5.7	2.1	virginica
## 126	7.2	3.2	6.0	1.8	virginica
## 127	6.2	2.8	4.8	1.8	virginica
## 128	6.1	3.0	4.9	1.8	virginica
## 129	6.4	2.8	5.6	2.1	virginica
## 130	7.2	3.0	5.8	1.6	virginica
## 131	7.4	2.8	6.1	1.9	virginica
## 132	7.9	3.8	6.4	2.0	virginica
## 133	6.4	2.8	5.6	2.2	virginica
## 134	6.3	2.8	5.1	1.5	virginica
## 135	6.1	2.6	5.6	1.4	virginica
## 136	7.7	3.0	6.1	2.3	virginica
## 137	6.3	3.4	5.6	2.4	virginica
## 138	6.4	3.1	5.5	1.8	virginica
## 139	6.0	3.0	4.8	1.8	virginica
## 140	6.9	3.1	5.4	2.1	virginica
## 141	6.7	3.1	5.6	2.4	virginica
## 142	6.9	3.1	5.1	2.3	virginica
## 143	5.8	2.7	5.1	1.9	virginica
## 144	6.8	3.2	5.9	2.3	virginica
## 145	6.7	3.3	5.7	2.5	virginica
## 146	6.7	3.0	5.2	2.3	virginica
## 147	6.3	2.5	5.0	1.9	virginica
## 148	6.5	3.0	5.2	2.0	virginica
## 149	6.2	3.4	5.4	2.3	virginica
## 150	5.9	3.0	5.1	1.8	virginica

2.17.2 More Visualization of the iris dataset

These are basic R functions that allow us to get general information about a dataframe:

- `dim()`
- `head()`
- `View()`
- `class()`
- `str()`

2.17.3 Basic R functions

```
dim(iris)
head(iris)
View(iris)
class(iris)
str(iris)
```

- Select the number of lines to visualize with the head command

```
head(iris, n = 10)
```

```
##      Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           5.1         3.5          1.4          0.2  setosa
## 2           4.9         3.0          1.4          0.2  setosa
## 3           4.7         3.2          1.3          0.2  setosa
## 4           4.6         3.1          1.5          0.2  setosa
## 5           5.0         3.6          1.4          0.2  setosa
## 6           5.4         3.9          1.7          0.4  setosa
## 7           4.6         3.4          1.4          0.3  setosa
## 8           5.0         3.4          1.5          0.2  setosa
## 9           4.4         2.9          1.4          0.2  setosa
## 10          4.9         3.1          1.5          0.1  setosa
```

- Determine the number of columns and rows in the dataframe:

```
dim(iris)
```

```
## [1] 150   5
```

- Determine the `class()` that the dataset belongs to

```
class(iris)
```

```
## [1] "data.frame"
```

2.18 More Visualization

2.18.1 Scatterplot (includes warnings)

```
plot(data=iris, iris$Sepal.Length, iris$Sepal.Width) ## R will complain about this command
```

```
## Warning in plot.window(...): "data" is not a graphical parameter
```

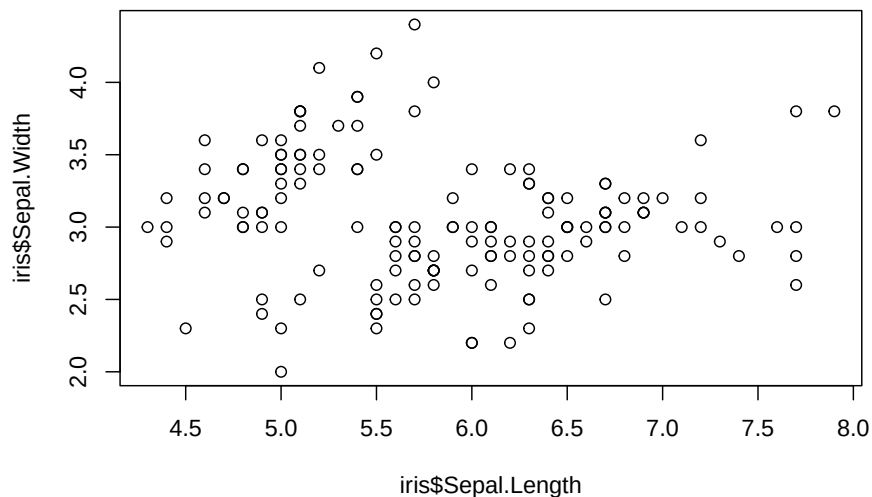
```
## Warning in plot.xy(xy, type, ...): "data" is not a graphical parameter
```

```
## Warning in axis(side = side, at = at, labels = labels, ...): "data" is not a  
## graphical parameter
```

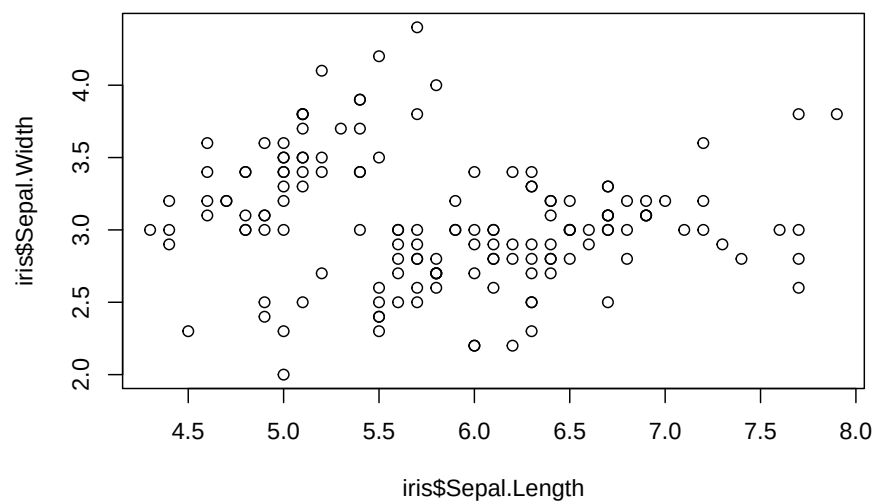
```
## Warning in axis(side = side, at = at, labels = labels, ...): "data" is not a  
## graphical parameter
```

```
## Warning in box(...): "data" is not a graphical parameter
```

```
## Warning in title(...): "data" is not a graphical parameter
```



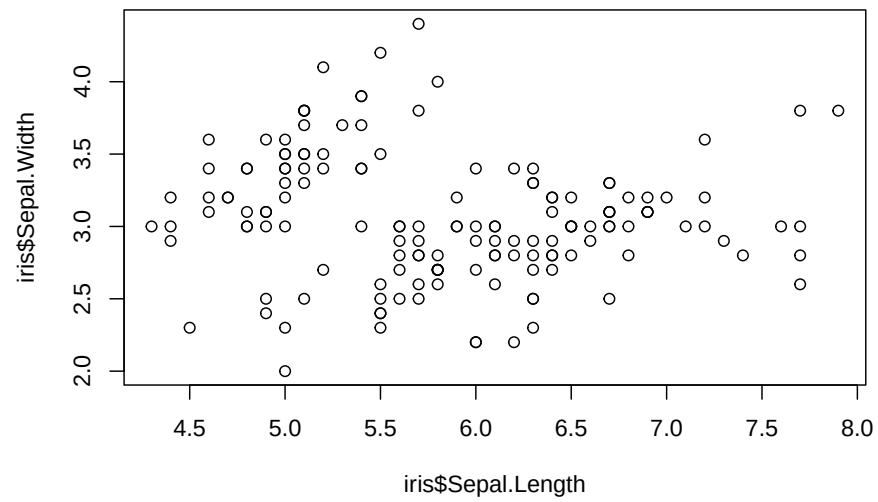
```
plot(iris$Sepal.Length, iris$Sepal.Width) ## According to the error, this was plotted using a dif
```



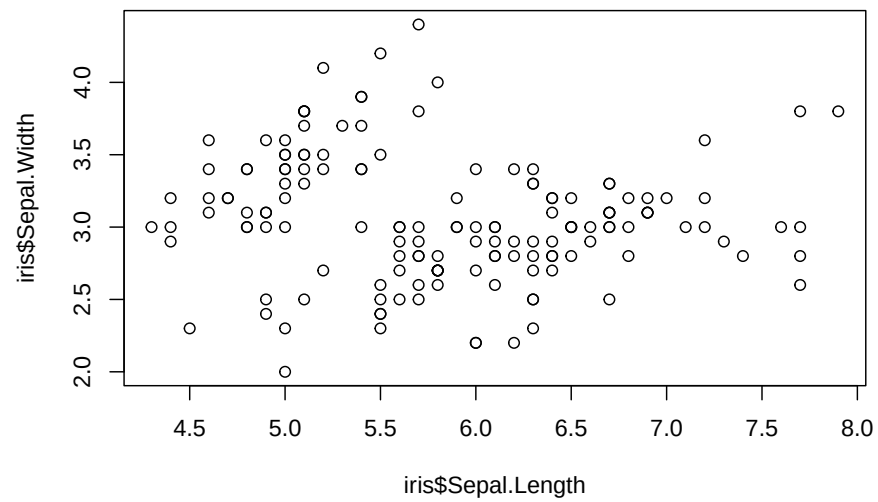
2.18.2 Scatterplot

Plot without warning and message

```
plot(data=iris, iris$Sepal.Length, iris$Sepal.Width) ## R will complain about this command
```

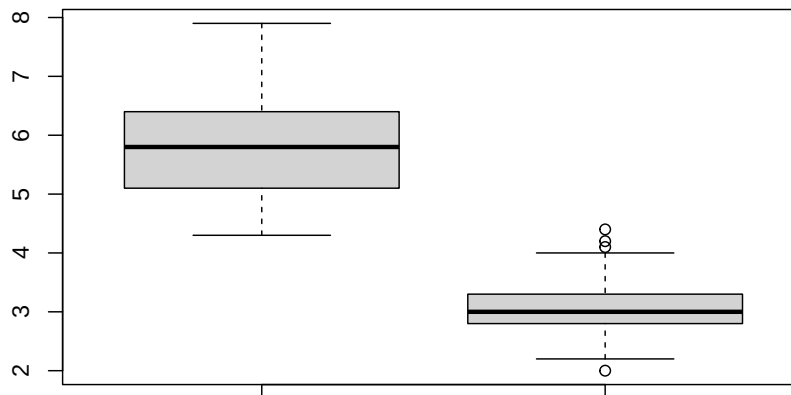


```
plot(iris$Sepal.Length, iris$Sepal.Width) ## According to the error, this was plotted
```



2.18.3 Boxplot

```
boxplot(data=iris, iris$Sepal.Length, iris$Sepal.Width)
```

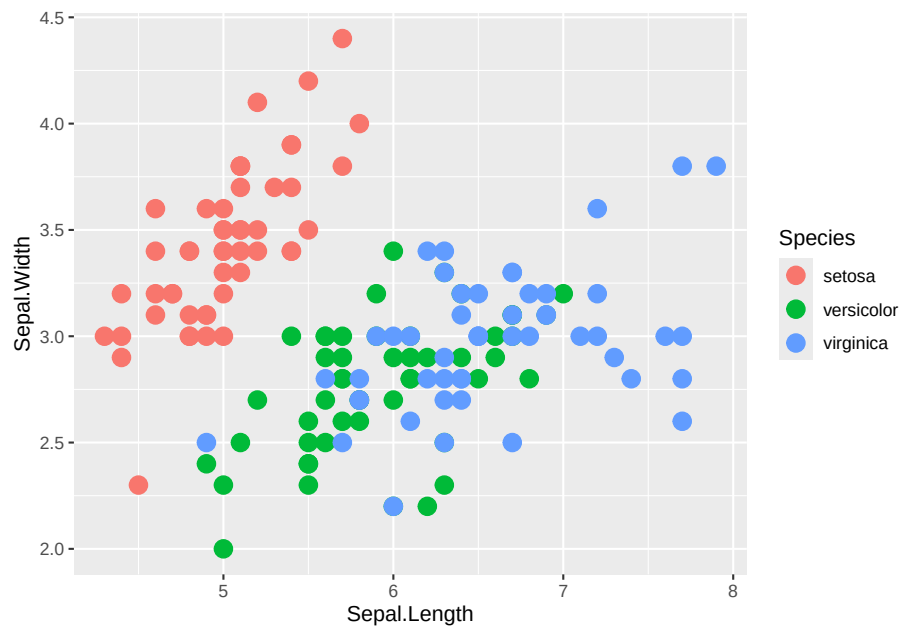


2.19 Data Visualization with specialized libraries

- In R, there are packages designed with the purpose of making good-looking graphics. This is the case with the ggplot2. The chunk below installs the ggplot2 library, loads the library into the R environment and then plots the data present in the iris data-frame.
- You can uncomment the installation line if you need to install it

2.19.1 Ggplot2

```
#install.packages("ggplot2")
library(ggplot2)
ggplot(data=iris, aes(x=Sepal.Length, y=Sepal.Width, color=Species)) + geom_point(size=4)
```



In our activities to visualize human genomic data, we will use a library called qqman, to visualize the biological association, through a plot known as the Manhattan plot.

2.19.2 Visualizing GWAS

- The simplest definition of a GWAS is the statistical or significant association between a phenotype (trait) and a genotype. This association can also be called *biological association*.
- Information about association of SNPs with Huntington's Disease can be found at the Chaves 2019 Huntington's disease paper

2.19.2.1 Package installation

- We need to install and load package qqman

```
## install.packages("qqman")
library(qqman)
```

```
##
```

```
## For example usage please run: vignette('qqman')
```

```
##
```

```
## Citation appreciated but not required:
```

```
## Turner, (2018). qqman: an R package for visualizing GWAS results using Q-Q and manhattan plots
```

```
##
```

2.19.2.2 Assign text file to dataframe object

- After, we load data-frame to be visualized
- Exact location of text file in your system needs to be determined

```
GWAS_TABLE <- read.table("./data/chr4.txt",
                          sep = " ",
                          header = T)
```

2.19.2.3 Get information about object with head()

- Use head() function to inspect the data-frame just loaded

```
head(GWAS_TABLE, n = 10)
```

```
##      CHR      SNP      BP      P
## 1      4 chr4:128096 128096 0.0133500
## 2      4 chr4:516586 516586 0.0076260
## 3      4 chr4:523979 523979 0.0024960
## 4      4 chr4:527217 527217 0.0217400
## 5      4 chr4:566177 566177 0.0008988
## 6      4 chr4:578679 578679 0.0162100
## 7      4 chr4:578790 578790 0.0103700
## 8      4 chr4:579307 579307 0.0334600
## 9      4 chr4:580259 580259 0.0190400
## 10     4 chr4:585318 585318 0.0317600
```

2.19.2.4 Plotting GWAS data-frame

- In this section we use functions plot() and boxplot() to visualize data-frame
- In the y access, we see genomic coordinates and in the x access, p-valoues of the biological association

- Note that depending on the syntax used for plotting, R may complain
- Comments not turned off

```
plot(data=GWAS_TABLE, GWAS_TABLE$BP, GWAS_TABLE$P) ## R complains
```

```
## Warning in plot.window(...): "data" is not a graphical parameter
```

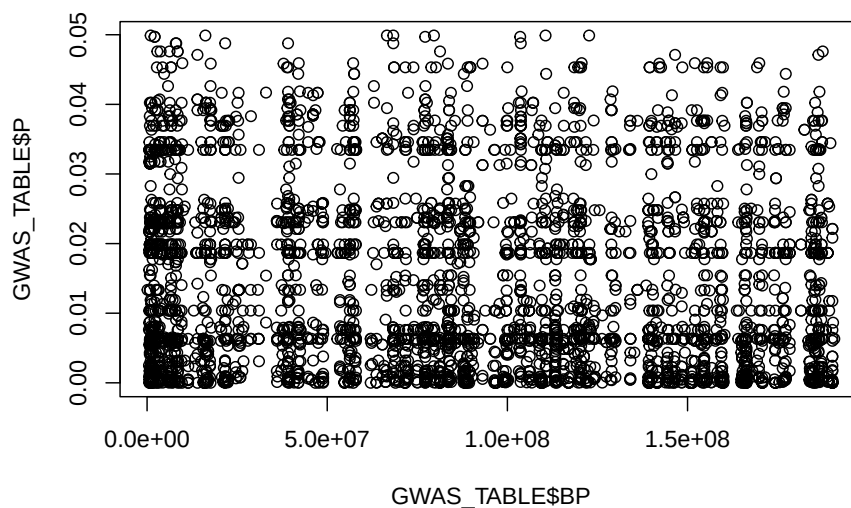
```
## Warning in plot.xy(xy, type, ...): "data" is not a graphical parameter
```

```
## Warning in axis(side = side, at = at, labels = labels, ...): "data" is not a  
## graphical parameter
```

```
## Warning in axis(side = side, at = at, labels = labels, ...): "data" is not a  
## graphical parameter
```

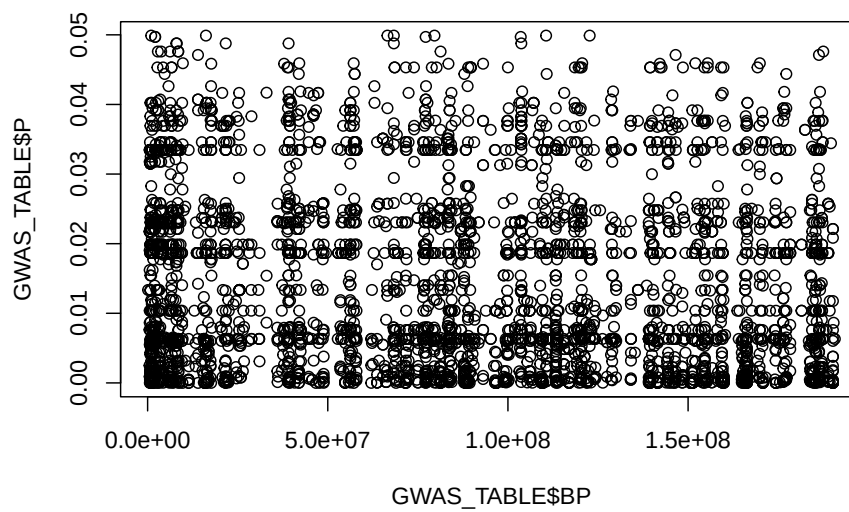
```
## Warning in box(...): "data" is not a graphical parameter
```

```
## Warning in title(...): "data" is not a graphical parameter
```



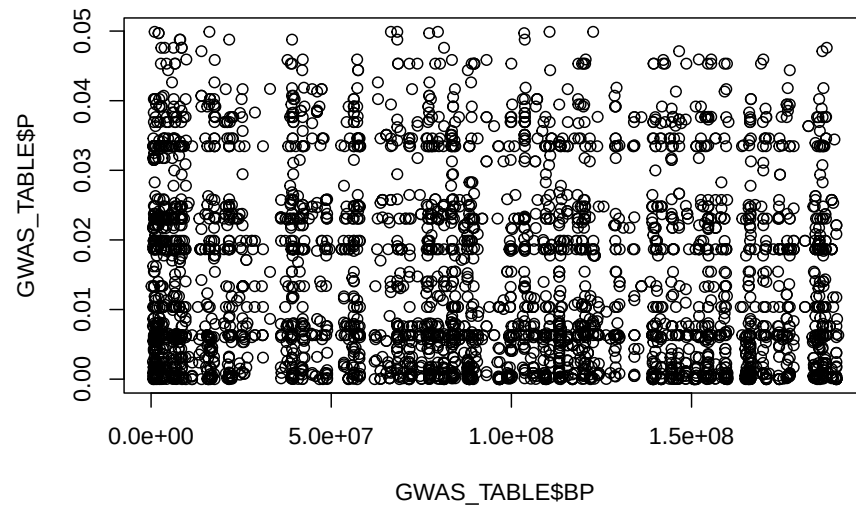
- Turn off comments

```
plot(data=GWAS_TABLE, GWAS_TABLE$BP, GWAS_TABLE$P) ## R complains
```



2.19.2.5 Plotting GWAS data-frame

```
plot(GWAS_TABLE$BP, GWAS_TABLE$P) ## R does not complain
```

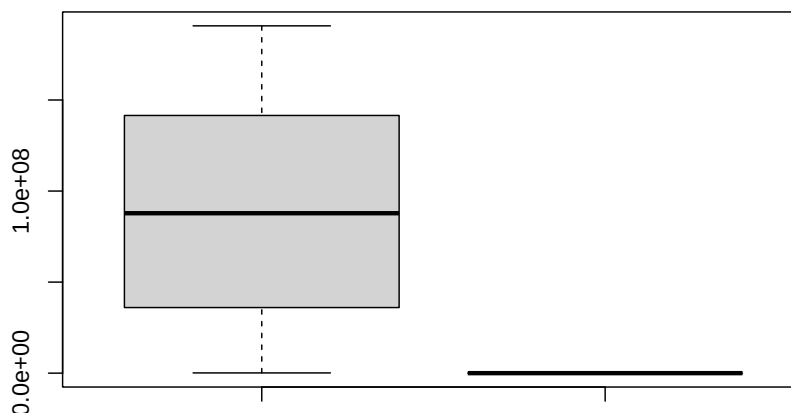


2.19.2.6 Boxplot

- Boxplot

```
boxplot(data=GWAS_TABLE, GWAS_TABLE$BP, GWAS_TABLE$P)
```

2.20. CREATE VECTOR TO HIGHLIGHT GENOME COORDINATES IN MANHATTAN PLOT31



- Compare the chromosomal coordinates and the values of the p-values in the boxplot above
- The chunk below allows one to omit NA values in data-frame

```
GWAS_TABLE_Ommit <- na.omit(GWAS_TABLE)
```

A seguir, criamos uma variável que armazena as posições das SNPs a serem realçadas em verde no Manhattan plot. Estas SNPs são mutações biológica e estatisticamente associadas à Doença de Huntington no gene de uma sortilina, localizada em proximidade física ao gene da proteína huntingtina mutada, a qual é a causadora medeliana (segue as leis de segregação genética de Mendel) da Doença de Huntington.

2.20 Create vector to highlight genome coordinates in Manhattan plot

```
SNP_HIGHLIGHT <- c("chr4:3043512", "chr4:3043513", "chr4:3048207", "chr4:3224216",  
  "chr4:3231772", "chr4:3233844", "chr4:3235081", "chr4:3235084",  
  "chr4:3236881", "chr4:3236883", "chr4:3241845", "chr4:3243804",  
  "chr4:3263138", "chr4:3265130", "chr4:3265710", "chr4:3314646",
```

```
"chr4:3380088", "chr4:3409359", "chr4:3411110", "chr4:3415336",  
"chr4:3415378", "chr4:3438643", "chr4:3446091", "chr4:3449886",  
"chr4:3473066", "chr4:3476809", "chr4:3480439", "chr4:3487151",  
"chr4:3496058", "chr4:3496110", "chr4:3506933", "chr4:3508752",  
"chr4:3510957", "chr4:3512690", "chr4:3517746", "chr4:3518190",  
"chr4:3529671", "chr4:3532327", "chr4:3533066", "chr4:3746133",  
"chr4:3747842", "chr4:3748134", "chr4:3765305", "chr4:3765336",  
"chr4:3944253", "chr4:3944752", "chr4:3944888", "chr4:3946166",  
"chr4:3946175", "chr4:3969218", "chr4:4051294", "chr4:4076788",  
"chr4:4103104", "chr4:4103105", "chr4:4109198", "chr4:4109210",  
"chr4:4240627", "chr4:4242705", "chr4:4243668", "chr4:4245210",  
"chr4:4245510", "chr4:4245513", "chr4:4245591", "chr4:4245926",  
"chr4:4245929", "chr4:4246109", "chr4:4246433", "chr4:4246453",  
"chr4:4246457", "chr4:4246497", "chr4:4249414", "chr4:4249415",  
"chr4:4249484", "chr4:4271623", "chr4:4275306", "chr4:4304749",  
"chr4:4318931", "chr4:4318970", "chr4:4319564", "chr4:4319728",  
"chr4:4319750", "chr4:4322078", "chr4:4709657", "chr4:4732282",  
"chr4:4789635", "chr4:4822960", "chr4:4824890", "chr4:4825092",  
"chr4:4825180", "chr4:4865316", "chr4:4865321", "chr4:5018702",  
"chr4:5812778", "chr4:5814082", "chr4:5833660", "chr4:5833899",  
"chr4:5835541", "chr4:5851205", "chr4:5862752", "chr4:5862938",  
"chr4:5862943", "chr4:5901873", "chr4:5905499", "chr4:5906287",  
"chr4:6018891", "chr4:6019046", "chr4:6020190", "chr4:6020367",  
"chr4:6025638", "chr4:6025656", "chr4:6025766", "chr4:6026058",  
"chr4:6083488", "chr4:6204935", "chr4:6235553", "chr4:6237142",  
"chr4:6238466", "chr4:6239906", "chr4:6240929", "chr4:6245022",  
"chr4:6245618", "chr4:6245732", "chr4:6245915", "chr4:6246075",  
"chr4:6246373", "chr4:6246959", "chr4:6290594", "chr4:6292020",  
"chr4:6294095", "chr4:6298375", "chr4:6316092", "chr4:6321396",  
"chr4:6324647", "chr4:6324785", "chr4:6327669", "chr4:6328354",  
"chr4:6328507", "chr4:6333130", "chr4:6333559", "chr4:6333669",  
"chr4:6335966", "chr4:6435341", "chr4:6435486", "chr4:6435926",  
"chr4:6437191", "chr4:6437197", "chr4:6457121", "chr4:6457131",  
"chr4:6457132", "chr4:6568390", "chr4:6570032", "chr4:6570768",  
"chr4:6596360", "chr4:6613252", "chr4:6613462", "chr4:6620991",  
"chr4:6624771", "chr4:6626154", "chr4:6641969", "chr4:6642090",  
"chr4:6644466", "chr4:6644467", "chr4:6644468", "chr4:6647889",  
"chr4:6648300", "chr4:6662665", "chr4:6663319", "chr4:6663715",  
"chr4:6674554", "chr4:6678553", "chr4:6678599", "chr4:6690535",  
"chr4:6698664", "chr4:6698667", "chr4:6698706", "chr4:6720572",  
"chr4:6911679", "chr4:6985889", "chr4:6987394", "chr4:7002344",  
"chr4:7004495", "chr4:7004506", "chr4:7005196", "chr4:7005199",  
"chr4:7024077", "chr4:7024398", "chr4:7029430", "chr4:7031064",  
"chr4:7044357", "chr4:7044380", "chr4:7048842", "chr4:7052115",  
"chr4:7055253", "chr4:7064243", "chr4:7067765", "chr4:7073187",
```

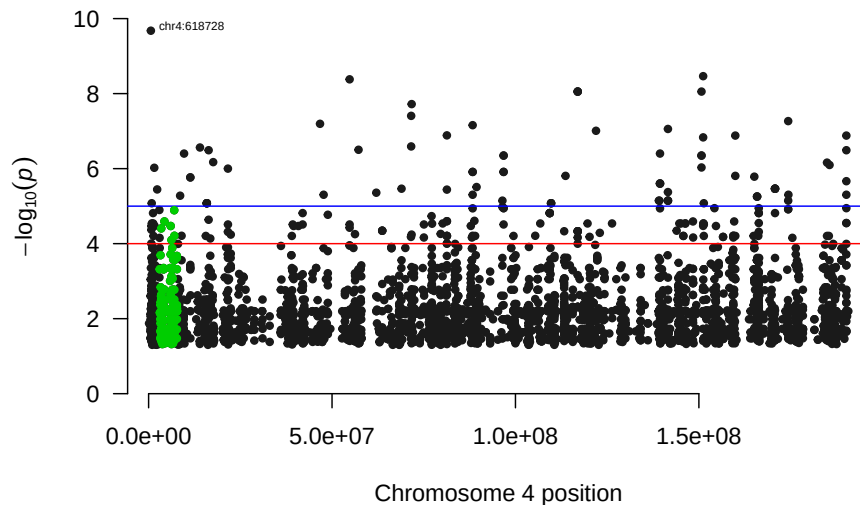


```
"chr4:7074027", "chr4:7677967", "chr4:7701947", "chr4:7702795",  
"chr4:7703505", "chr4:7703807", "chr4:7704795", "chr4:7704818",  
"chr4:7709703", "chr4:7712150", "chr4:7714490", "chr4:7733843",  
"chr4:7735162", "chr4:7735164", "chr4:7736103", "chr4:7736112")
```

2.21 Biological Association Visualization: Manhattan Plot

- Finally, we plot the Manhattan plot graph

```
manhattan(GWAS_TABLE_Ommit,  
          highlight = SNP_HIGHLIGHT,  
          annotateTop = T,  
          annotatePval = 0.20,  
          genomewideline = -log10(0.0001))
```



2.22 References

2.23 Session Info

```
sessionInfo()
```

```
## R version 4.4.1 (2024-06-14)
## Platform: aarch64-apple-darwin20
## Running under: macOS 15.7.4
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Chicago
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] qqman_0.1.9  ggplot2_3.5.1
##
## loaded via a namespace (and not attached):
##  [1] vctrs_0.6.5      cli_3.6.3        knitr_1.48       rlang_1.1.4
##  [5] xfun_0.48        highr_0.11       generics_0.1.3   calibrate_1.7.7
##  [9] labeling_0.4.3   glue_1.8.0       colorspace_2.1-1 htmltools_0.5.8.1
## [13] tinytex_0.53     scales_1.3.0     fansi_1.0.6      rmarkdown_2.28
## [17] grid_4.4.1       evaluate_1.0.0   munsell_0.5.1    tibble_3.2.1
## [21] MASS_7.3-61      fastmap_1.2.0    yaml_2.3.10      lifecycle_1.0.4
## [25] bookdown_0.40    compiler_4.4.1   dplyr_1.1.4      pkgconfig_2.0.3
## [29] rstudioapi_0.16.0 farver_2.1.2     digest_0.6.37    R6_2.5.1
## [33] tidyselect_1.2.1 utf8_1.2.4       pillar_1.9.0     magrittr_2.0.3
## [37] withr_3.0.1      tools_4.4.1      gtable_0.3.5
```

Chapter 3

Processamento de dados públicos de neuroblastoma

3.1 Origem dos dados

Os dados analisados neste módulo têm origem na base de dados genômicos R2.

A plataforma R2 é uma ferramenta de análise e visualização genômica online de livre acesso que pode analisar uma grande coleção de dados públicos, mas também permite a análise protegida de conjuntos de dados privados, conforme indicado por Rijswijk (2024).

A R2 consiste em um banco de dados que armazena as informações genômicas, acoplado a um amplo conjunto de ferramentas para analisar/visualizar os conjuntos de dados.

Mais informações sobre a plataforma R2 pode ser encontradas no site da plataforma:

<https://hgserver1.amc.nl/cgi-bin/r2/main.cgi>

3.2 Carregando dados pre-processados de R2

Nesta parte, vamos carregar uma dataframe contendo dados baixados de R2 em que já fizemos pre-processamento. Vamos gerar algumas visualizações para

nos familiarizarmos com os dados. O objeto `r2_gse62564_GSVA_Metadata.rds` deve ser baixado da pasta no Google Drive no seguinte link: <https://drive.google.com/drive/folders/1Kkos4HK2EP4ntrQOJE0mrHYlpeMa1yi4?usp=sharing>

Uma vez que o arquivo é baixado, pode-se carregá-lo no ambiente R usando a função `readRDS`, como feito abaixo:

```
r2_gse62564_GSVA_Metadata <- readRDS("./data/r2_gse62564_GSVA_Metadata.rds")
```

No neuroblastoma, o principal sistema de classificação de risco classifica os pacientes usando características como idade, estadiamento, histologia e status de amplificação do MYCN.

O prognóstico é a associação entre as características do tumor e a sobrevida do paciente.

O MYCN é um gene que codifica um fator de transcrição muito importante no prognóstico do tumor.

Vamos analisar a expressão do MYCN comparada pelo grupo de risco do neuroblastoma.

Primeiro, vamos ver que tipo de variável é o status do MYCN e a expressão do MYCN.

Verificar a classe da variável `mycn_status`:

```
class(r2_gse62564_GSVA_Metadata$mycn_status)
```

```
## [1] "character"
```

Verificar a classe da variável MYCN:

```
class(r2_gse62564_GSVA_Metadata$MYCN)
```

```
## [1] "character"
```

Ambos são do tipo “character”. Para plotar, precisamos fazer com que a expressão MYCN seja do tipo numérico:

```
r2_gse62564_GSVA_Metadata$MYCN <- as.numeric(
  r2_gse62564_GSVA_Metadata$MYCN)
```

Verificar novamente a classe da variável MYCN:

```
class(r2_gse62564_GSVA_Metadata$MYCN)
```

```
## [1] "numeric"
```

Agora a classe de MYCN é um vetor numérico.

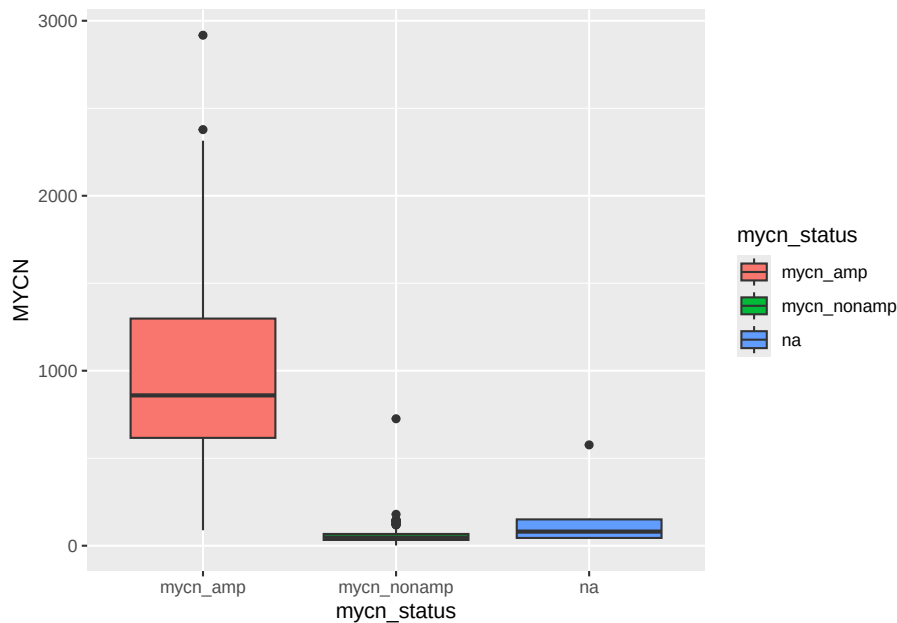
3.3 Primeiros gráficos com dados de neuroblastoma

3.3.1 Mycn_status vs Expressão de MYCN

Vamos plotar uma figura com a variável mycn_status no eixo X e expressão de MYCN no eixo Y.

Vamos tentar ver se o ggplot, que usamos antes, considera a variável mycn_status categórica como ela é agora.

```
library(ggplot2)
ggplot(r2_gse62564_GSVA_Metadata, aes(x=mycn_status, y=MYCN,
                                       fill=mycn_status)) +
  geom_boxplot()
```

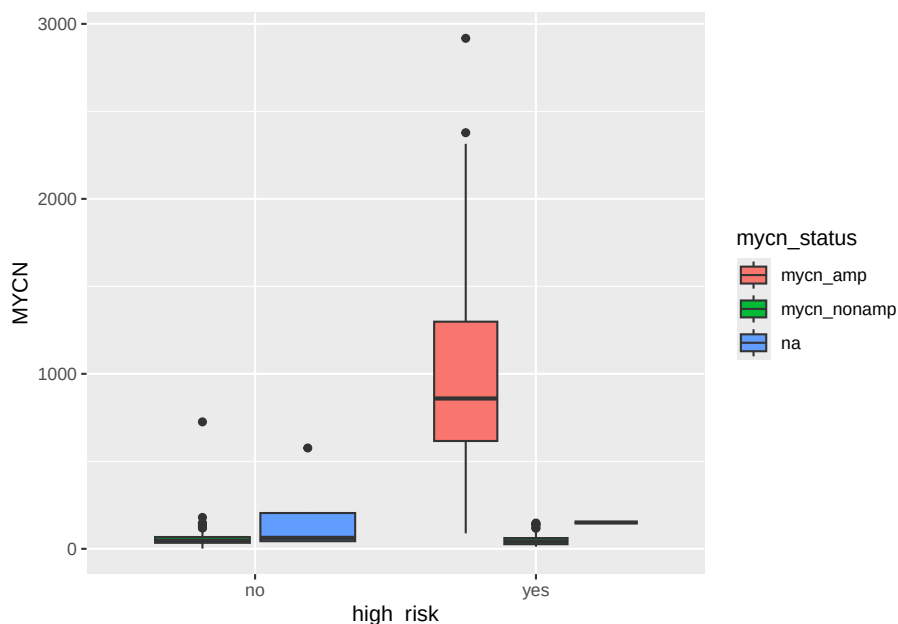


Vemos que existem três categorias de `mycn_status`: `mycn_amp`, `mycn_nonamp` e `na`. `mycn_amp` significa amplificação de MYCN, um fator de mau prognóstico. `mycn_nonamp` significa ausência de amplificação de MYCN. `na` significa que não há informações sobre o status de MYCN. Vemos que a amplificação de MYCN está altamente associada à expressão de MYCN, no eixo Y. Por enquanto, não vamos nos preocupar nem com a formatação dos rótulos X e Y nem com os valores N/A do status de amplificação MYCN.

3.3.2 Alto Risco vs expressão de MYCN

Podemos plotar o status de alto risco vs. amplificação MYCN e deixar o argumento de preenchimento como `mycn_status`

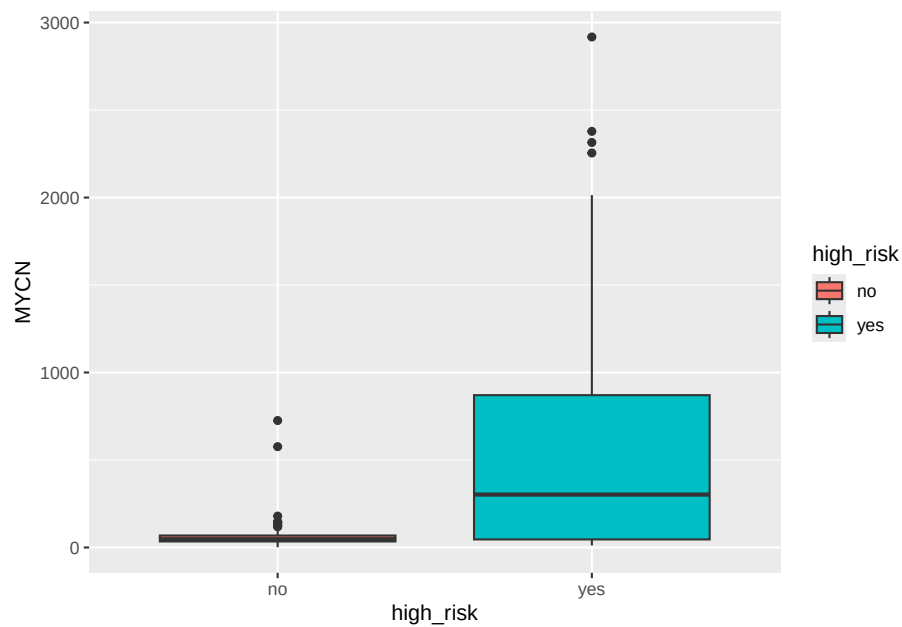
```
library(ggplot2)
ggplot(r2_gse62564_GSVA_Metadata, aes(x=high_risk, y=MYCN,
                                       fill=mycn_status)) +
  geom_boxplot()
```



Aqui vemos que a maior parte dos pacientes de alto risco (High Risk) apresentam também amplificação de MYCN e que nesses pacientes, a expressão de MYCN é alta.

Podemos ainda querer ver apenas a expressão de MYCN no grupo de alto risco, usando o parâmetro `fill=high_risk`:

```
library(ggplot2)
ggplot(r2_gse62564_GSVA_Metadata, aes(x=high_risk, y=MYCN,
                                       fill=high_risk)) +
  geom_boxplot()
```



Onde confirmamos que a expressão de MYCN é maior nos pacientes de alto risco (High Risk).

Chapter 4

Análise de Sobrevivência

Na análise de sobrevivência estudamos como varia a probabilidade de sobrevivência dos pacientes em função do tempo transcorrido desde o início de um estudo. Na atividade proposta para este módulo, vamos avaliar como é a probabilidade de sobrevivência de pacientes de neuroblastoma em função do status de alto risco (se um paciente é de alto risco ou baixo risco). Para dar prosseguimento, carregamos a dataframe de neuroblastoma no ambiente R, como fizemos no módulo anterior:

```
r2_gse62564_GSVA_Metadata <- readRDS( "./data/r2_gse62564_GSVA_Metadata.rds")
```

4.1 Processamento de variáveis para permitir a construção de gráficos

Precisamos definir o status do OS (Overall Survival = sobrevivência geral) como numérico, em vez de um caractere. Fazemos o mesmo para o status EFS (Event Free Survival = sobrevivência livre de evento).

Usamos o método `ifelse` para construir `os_bin_numeric` a partir de `os_bin`, introduzindo números ao invés da palavra evento e não evento. No caso do Overall Survival (OS), a morte do paciente é o evento.

Para usar o comando a seguir, precisamos de usar, antes, uma biblioteca do R chamada `maditr`:

```
library(maditr)
```

```
##  
## To select rows from data: rows(mtcars, am==0)
```

```
##
## Attaching package: 'maditr'

## The following object is masked from 'package:base':
##
##      sort_by
```

Agora podemos usar o operador pipe (`%>%`), que toma o resultado de um comando e passa para o próximo comando:

```
r2_gse62564_GSVA_Metadata <- r2_gse62564_GSVA_Metadata %>%
  dplyr::mutate(os_bin_numeric =
    ifelse(r2_gse62564_GSVA_Metadata$os_bin == "event", "1", "0"))
```

Agora, fazemos `os_bin_numeric` numérico:

```
r2_gse62564_GSVA_Metadata["os_bin_numeric"] <- as.numeric(r2_gse62564_GSVA_Metadata$os_bin_numeric)
```

Fazemos o mesmo para EFS:

```
## EFS
r2_gse62564_GSVA_Metadata <- r2_gse62564_GSVA_Metadata %>%
  dplyr::mutate(efs_bin_numeric =
    ifelse(r2_gse62564_GSVA_Metadata$efs_bin == "event", "1", "0"))
```

Agora, fazemos `efs_bin_numeric`, numérico:

```
r2_gse62564_GSVA_Metadata["efs_bin_numeric"] <- as.numeric(r2_gse62564_GSVA_Metadata$efs_bin_numeric)
```

4.2 Fazendo a análise de sobrevivência

4.2.1 Construção da curva de sobrevivência

Usamos a função `survfit` do pacote em R, `survival`, para construir o objeto `fit_os_high_risk`, ajustando o modelo:

```
library(survival)
fit_os_high_risk <- survfit(Surv(as.numeric(os_day), os_bin_numeric) ~ r2_gse62564_GSVA_Metadata$efs_bin_numeric,
  data = r2_gse62564_GSVA_Metadata)
```

Observe que `os_day` é uma variável contendo o número de dias até o evento de interesse.

4.2.2 Plotagem da curva de sobrevivência

Usamos a função `ggsurvplot` do pacote em R `survminer` para plotar a curva de sobrevivência:

```
library(survminer)
```

```
## Loading required package: ggpubr
```

```
##
```

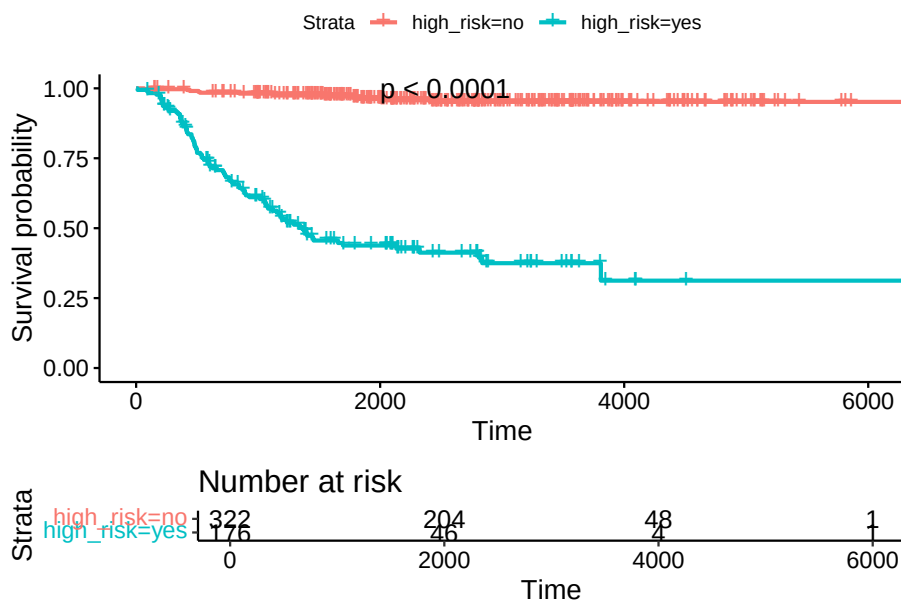
```
## Attaching package: 'survminer'
```

```
## The following object is masked from 'package:survival':
```

```
##
```

```
##      myeloma
```

```
ggsurvplot(fit_os_high_risk,
  data = r2_gse62564_GSVA_Metadata,
  risk.table = TRUE,
  # risk.table = "abs_pct",
  pval = TRUE,
  pval.coord = c(2000, 1),
)
```



4.2.4 Análise de sobrevivência do gene da hipóxia

Como vemos que indivíduos de alto risco apresentam maior expressão de HIF1A, vamos analisar também a análise de sobrevivência de HIF1A. Para isso vamos separar em dois grupos de pacientes, baseados na expressão do HIF1A. Construindo a variável HIF1A em dois quantis, um com alta expressão de HIF1A e um com baixa expressão de HIF1A. Para isso vamos usar a função `ntile()` do pacote `tidyverse`:

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v lubridate  1.9.3      v tibble    3.2.1
## v purrr      1.0.2      v tidyr     1.3.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::between()      masks maditr::between()
## x dplyr::coalesce()     masks maditr::coalesce()
## x readr::cols()         masks maditr::cols()
## x dplyr::filter()       masks stats::filter()
## x dplyr::first()        masks maditr::first()
## x dplyr::lag()          masks stats::lag()
## x dplyr::last()         masks maditr::last()
## x purrr::transpose()    masks maditr::transpose()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
r2_gse62564_GSVA_Metadata <- r2_gse62564_GSVA_Metadata %>%
  mutate(quantile = ntile(HIF1A, 2))
```

Fazendo da variável quantil HIF1A um fator

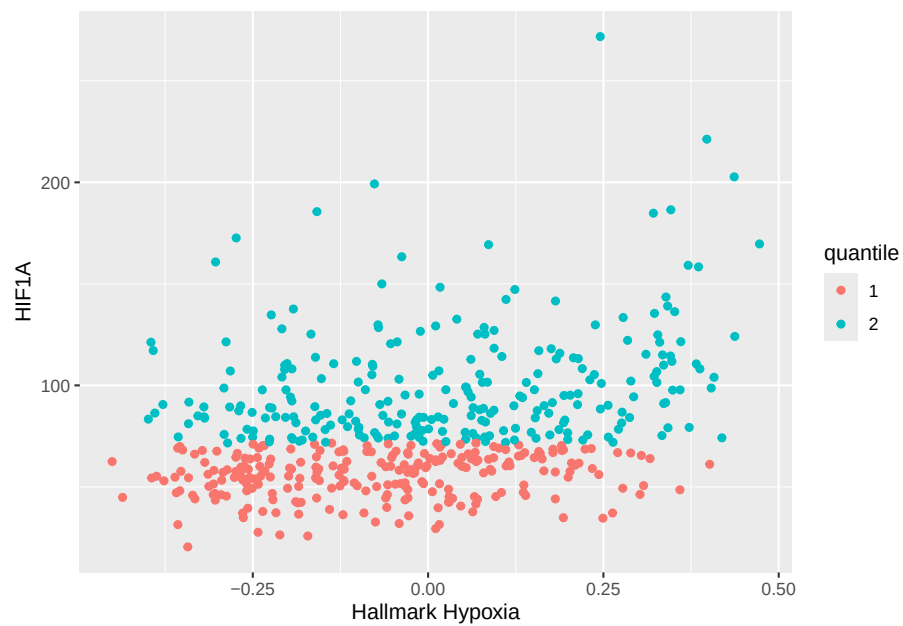
```
r2_gse62564_GSVA_Metadata$quantile <- as.factor(r2_gse62564_GSVA_Metadata$quantile )
```

Gráfico de dispersão HALLMARK HYPOXIA vs. HIF1A

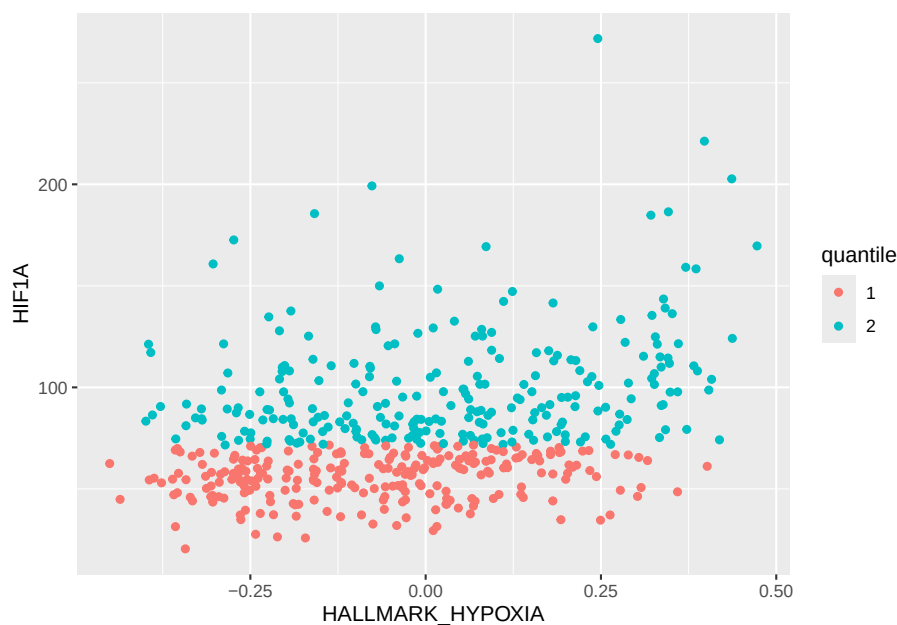
```
r2_gse62564_GSVA_Metadata$HALLMARK_HYPOXIA <- as.numeric(r2_gse62564_GSVA_Metadata$HALLMARK_HYPOXIA)

qplot(HALLMARK_HYPOXIA, HIF1A,
      data = r2_gse62564_GSVA_Metadata,
      colour=quantile,
      ylab = "HIF1A",
      xlab = "Hallmark Hypoxia")
```

```
## Warning: `qplot()` was deprecated in ggplot2 3.4.0.  
## This warning is displayed once every 8 hours.  
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was  
## generated.
```



```
library(ggplot2)  
# Basic scatter plot  
ggplot(r2_gse62564_GSVA_Metadata, aes(x=HALLMARK_HYPOXIA, y=HIF1A, color=quantile)) +
```



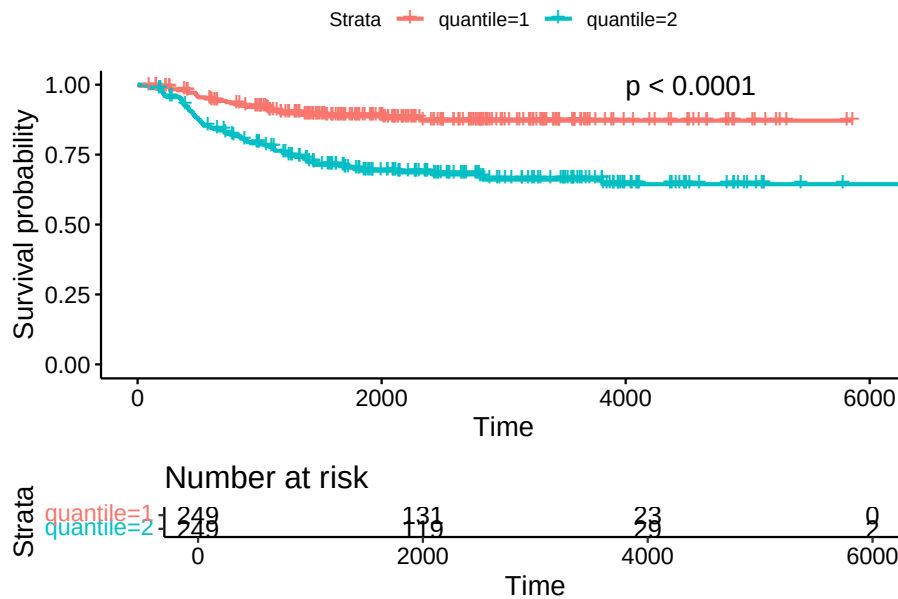
Vemos que o quantil 2 tem valores de expressão de HIF1A maiores que os valores do quantil 1.

Construímos o objeto de sobrevivência ajustado com a variável quantil HIF1A:

```
library(survival)
fit_os_HIF1A <- survfit(Surv(as.numeric(os_day), os_bin_numeric) ~ r2_gse62564_GSVA_Metadata$quantil,
  data = r2_gse62564_GSVA_Metadata)
```

E finalmente traçamos a curva de sobrevivência do gráfico do Overall Survival:

```
library(survminer)
ggsurvplot(fit_os_HIF1A,
  data = r2_gse62564_GSVA_Metadata,
  risk.table = TRUE,
  # risk.table = "abs_pct",
  pval = TRUE,
  pval.coord = c(4000, 1),
)
```



Observamos que os maiores valores na expressão de HIF1A (segundo quantil ou quantil 2. No inglês é second quantile) levam a menor probabilidade de sobrevivência. Vemos que no dataset, o segundo quantil tem menores probabilidades de sobrevivência.

4.2.4.1 Event-free survival

Construímos a variável quantil HIF1A:

```
library(tidyverse)
r2_gse62564_GSVA_Metadata <- r2_gse62564_GSVA_Metadata %>%
  mutate(quantile = ntile(HIF1A, 2))
```

Construímos objeto de sobrevivência EFS, `fit_efs_HIF1A`

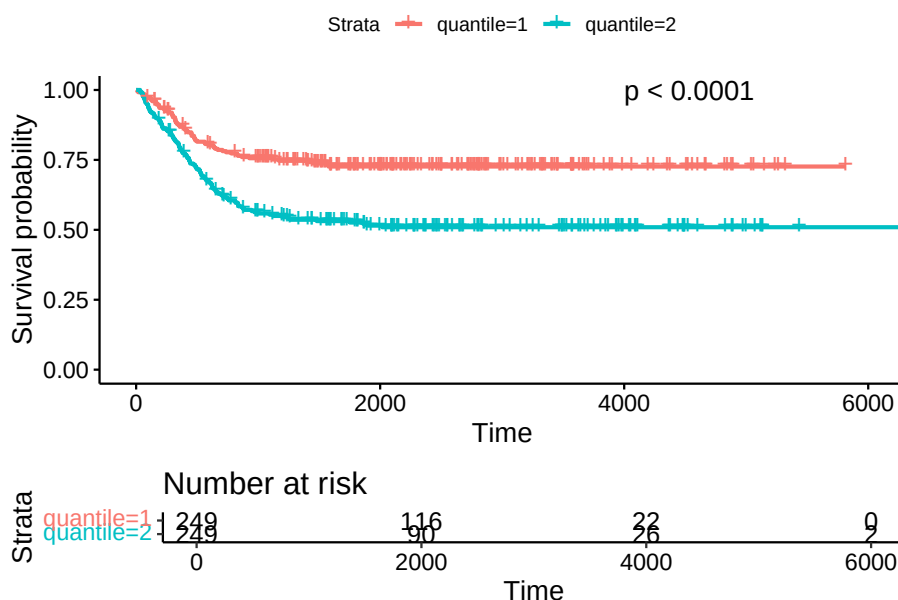
```
library(survival)
fit_efs_HIF1A <- survfit(Surv(as.numeric(efs_day), efs_bin_numeric) ~ r2_gse62564_GSVA_Metadata,
  data = r2_gse62564_GSVA_Metadata)
```

Curva de sobrevivência do gráfico EFS, usando a função `ggsurvplot`

```
library(survminer)
ggsurvplot(fit_efs_HIF1A,
```



```
data = r2_gse62564_GSVA_Metadata,
risk.table = TRUE,
# risk.table = "abs_pct",
pval = TRUE,
pval.coord = c(4000, 1),
)
```



4.2.5 Relação do HIF1A com a idade no diagnóstico

Precisamos criar o objeto `r2_gse62564_GSVA_Metadata_Spaces_survival`

A função `compare_means` não consegue lidar com espaços no nome da variável de exaustão de células T.

Teremos que usar um truque para lidar com os espaços em “T cell exhaustion”.

Usaremos o pacote `stringr` para inserir pontos (.) substituindo os espaços no dataframe `r2_gse62564_GSVA_Metadata`.

```
r2_gse62564_GSVA_Metadata_Spaces <- r2_gse62564_GSVA_Metadata
library(stringr)
names(r2_gse62564_GSVA_Metadata_Spaces) <- str_replace_all(names(r2_gse62564_GSVA_Metadata_Spaces),
```

Criando o objeto `r2_gse62564_GSVA_Metadata_Spaces_survival`:

```
r2_gse62564_GSVA_Metadata_Spaces_survival <- r2_gse62564_GSVA_Metadata_Spaces
```

Fazendo age_at_diagnosis numérico para plotar os quantis

```
r2_gse62564_GSVA_Metadata_Spaces_survival$age_at_diagnosis <- as.numeric(r2_gse62564_GSVA_Metadata_Spaces_survival$age_at_diagnosis)
```

Verificando a idade máxima no diagnóstico

```
max(r2_gse62564_GSVA_Metadata_Spaces_survival$age_at_diagnosis)
```

```
## [1] 8983
```

```
min(r2_gse62564_GSVA_Metadata_Spaces_survival$age_at_diagnosis)
```

```
## [1] 0
```

Construir quantis para a idade no diagnóstico (age_qtls)

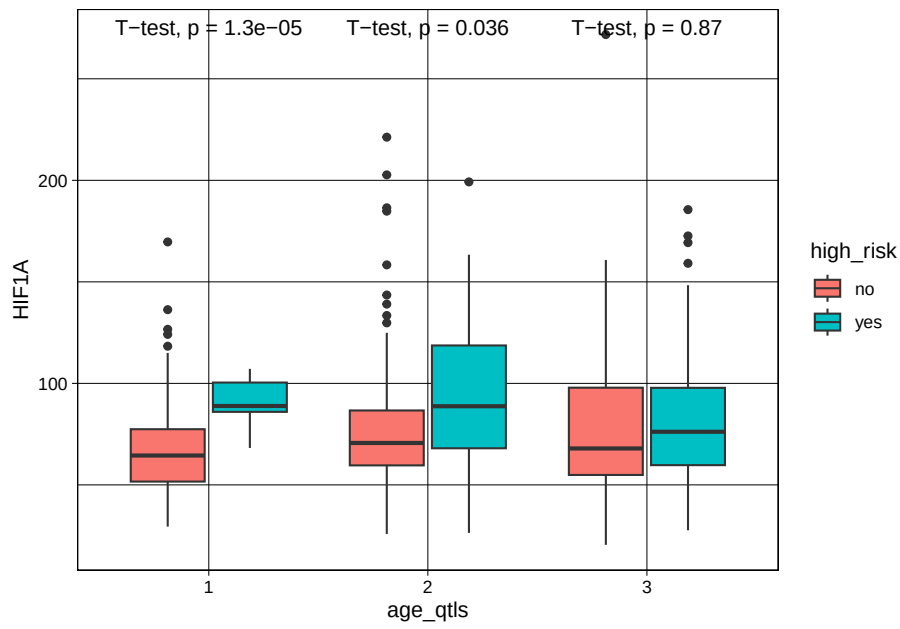
```
r2_gse62564_GSVA_Metadata_Spaces_survival_age <- r2_gse62564_GSVA_Metadata_Spaces_survival %>%  
  mutate(age_qtls = ntile(age_at_diagnosis, 3))
```

Faça age_qtls como fator

```
r2_gse62564_GSVA_Metadata_Spaces_survival_age$age_qtls <- as.factor(r2_gse62564_GSVA_Metadata_Spaces_survival_age$age_qtls)
```

Gráfico age_qtls vs. HIF1A

```
ggplot(r2_gse62564_GSVA_Metadata_Spaces_survival_age, aes(x=age_qtls, y=HIF1A, fill=age_qtls)) +  
  geom_boxplot() +  
  stat_compare_means(method = "t.test") +  
  theme_linedraw()
```



Chapter 5

Accessing the Raça/Cor Variable in DATASUS

5.1 Load libraries

```
library(microdatasus)
library(dplyr)
library(tidyverse)
library(maditr)
library(kableExtra)
```

5.2 Constrir dataframe (df) com identificação racial

```
racacor_df <- readRDS("./data/racacor_df.rds")
```

5.3 Transformar variável Raça/Cor

Transforma coluna com caracteres em nomes de raça/cor

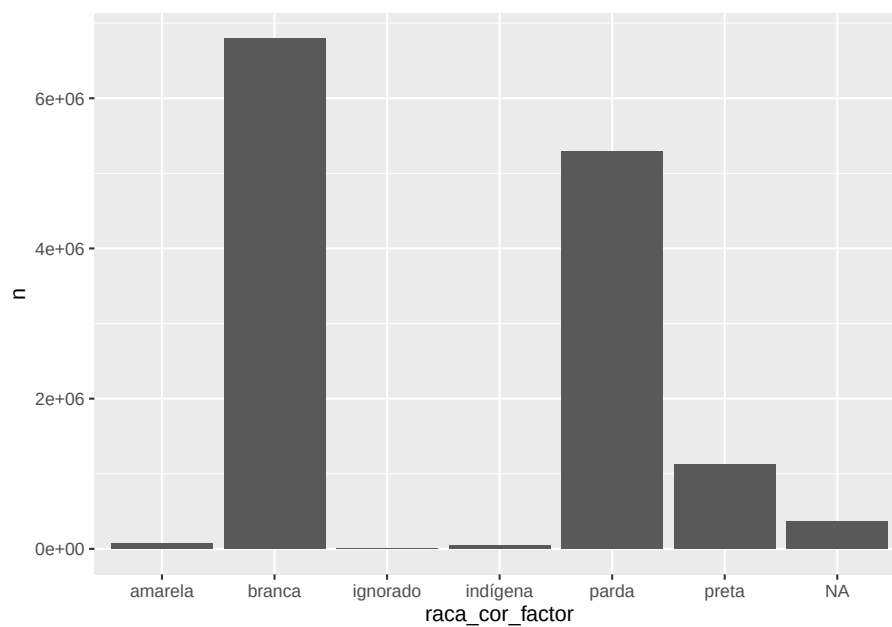
```
racacor_df <- racacor_df %>%  
  dplyr::mutate(raca_cor_factor = ifelse(racacor_df$RACACOR == "1", "branca",  
                                         ifelse(racacor_df$RACACOR == "2", "preta",  
                                                   ifelse(racacor_df$RACACOR == "3", "amarela",  
                                                         ifelse(racacor_df$RACACOR == "4", "indigena",  
                                                             ifelse(racacor_df$RACACOR == "5", "outros",  
                                                                 ifelse(racacor_df$RACACOR == "6", "sem_informacao",
```

5.4 Gráfico de barras por categoria da Variável Raça/Cor

Nesta parte, poderemos estimar o número de indivíduos sob risco, considerando as categorias raciais brasileiras.

```
library(ggplot2)
# Basic barplot
ggplot(data=racacor_df, aes(x=raca_cor_factor, y=n)) +
  geom_bar(stat="identity")
```

5.4. GRÁFICO DE BARRAS POR CATEGORIA DA VARIÁVEL RAÇA/COR55



```
dados_agregados_year <- readRDS("./data/dados_agregados_year.rds")
dados_agregados_year
```

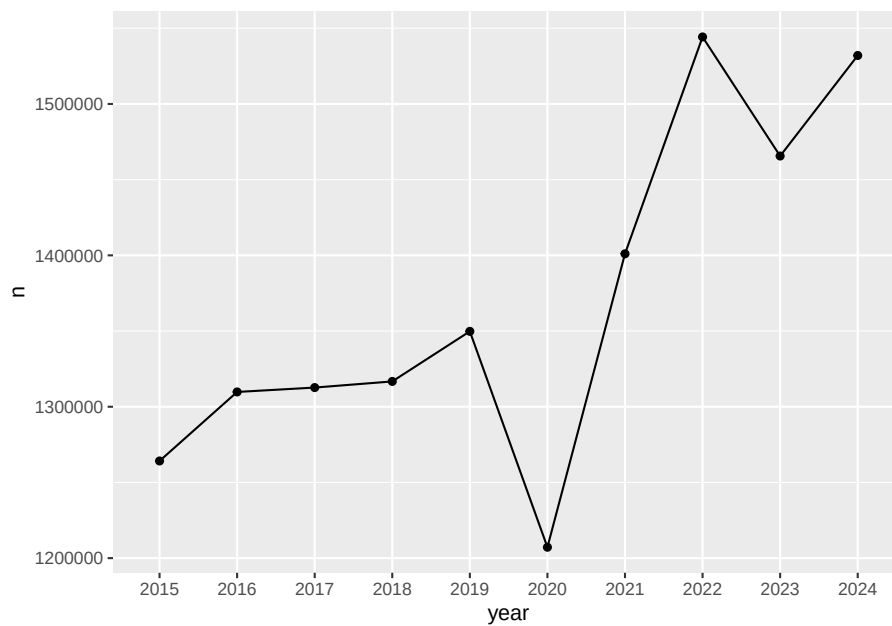
```
## # A tibble: 10 x 2
##   year      n
##   <chr>  <int>
## 1 2015 1264175
## 2 2016 1309774
## 3 2017 1312663
## 4 2018 1316719
## 5 2019 1349801
## 6 2020 1207189
## 7 2021 1401033
## 8 2022 1544266
## 9 2023 1465610
## 10 2024 1532015
```

Remover NAs dos dados agregados por ano

```
dados_agregados_year <- dados_agregados_year[ which(
  dados_agregados_year$year != "NA"), ]
```

5.5 Mortalidade Anual Total

```
library(ggplot2)
# Basic line plot with points
ggplot(data=dados_agregados_year, aes(x=year, y=n, group=1)) +
  geom_line() +
  geom_point()
```



5.5.1 Mortalidade Anual Total Usando Variável Raça/Cor

Mortalidade anual total (figura acima) e mortalidade anual usando a variável Raça/Cor (figura abaixo) foram feitas usando ? como referência.

Carregar dados_appended_melted

```
dados_appended_melted <- readRDS("./data/dados_appended_melted.rds")
dados_appended_melted
```

##	year	variable	value
## 1	2015	1	643928
## 2	2016	1	665737
## 3	2017	1	663147
## 4	2018	1	668707
## 5	2019	1	686474
## 6	2020	1	523848
## 7	2021	1	644210
## 8	2022	1	787363
## 9	2023	1	737590
## 10	2024	1	773683
## 11	2015	2	93830
## 12	2016	2	97747
## 13	2017	2	99745
## 14	2018	2	102889
## 15	2019	2	107306
## 16	2020	2	107171
## 17	2021	2	122852
## 18	2022	2	131005
## 19	2023	2	129047
## 20	2024	2	135916
## 21	2015	3	7051
## 22	2016	3	7276
## 23	2017	3	7375
## 24	2018	3	7524
## 25	2019	3	7359
## 26	2020	3	4716
## 27	2021	3	5313
## 28	2022	3	9261
## 29	2023	3	9053
## 30	2024	3	9914
## 31	2015	4	458807
## 32	2016	4	483026
## 33	2017	4	497839
## 34	2018	4	496106

```
## 35 2019      4 509936
## 36 2020      4 531808
## 37 2021      4 587323
## 38 2022      4 582469
## 39 2023      4 561438
## 40 2024      4 586563
## 41 2015      5   3640
## 42 2016      5   3867
## 43 2017      5   3911
## 44 2018      5   4261
## 45 2019      5   4273
## 46 2020      5   5173
## 47 2021      5   5361
## 48 2022      5   5343
## 49 2023      5   5398
## 50 2024      5   5555
## 51 2015      9     0
## 52 2016      9     5
## 53 2017      9     0
## 54 2018      9     0
## 55 2019      9     0
## 56 2020      9     0
## 57 2021      9     0
## 58 2022      9     0
## 59 2023      9     0
## 60 2024      9     0
## 61 2015     NA 56919
## 62 2016     NA 52116
## 63 2017     NA 40646
## 64 2018     NA 37232
## 65 2019     NA 34453
## 66 2020     NA 34473
## 67 2021     NA 35974
## 68 2022     NA 28825
## 69 2023     NA 23084
## 70 2024     NA 20384
```

Obter variável raça/cor como palavras descritivas ao invés de números.

```
dados_appended_melted <- dados_appended_melted %>%
  dplyr::mutate(raca_cor_factor = ifelse(dados_appended_melted$variable == "1", "branco",
                                         ifelse(dados_appended_melted$variable == "2",
                                                  ifelse(dados_appended_melted$variable == "3", "preto", "outro")
                                         )
  )
```

```

# Checar dados_appended_melted
dados_appended_melted

```

```

ifelse(dados_appended_melted$variable ==
ifelse(dados_appended_melted$variab
ifelse(dados_appended_melted

```

```

##   year variable  value raca_cor_factor
## 1  2015         1 643928          branca
## 2  2016         1 665737          branca
## 3  2017         1 663147          branca
## 4  2018         1 668707          branca
## 5  2019         1 686474          branca
## 6  2020         1 523848          branca
## 7  2021         1 644210          branca
## 8  2022         1 787363          branca
## 9  2023         1 737590          branca
## 10 2024         1 773683          branca
## 11 2015         2  93830          preta
## 12 2016         2  97747          preta
## 13 2017         2  99745          preta
## 14 2018         2 102889          preta
## 15 2019         2 107306          preta
## 16 2020         2 107171          preta
## 17 2021         2 122852          preta
## 18 2022         2 131005          preta
## 19 2023         2 129047          preta
## 20 2024         2 135916          preta
## 21 2015         3   7051          amarela
## 22 2016         3   7276          amarela
## 23 2017         3   7375          amarela
## 24 2018         3   7524          amarela
## 25 2019         3   7359          amarela
## 26 2020         3   4716          amarela
## 27 2021         3   5313          amarela
## 28 2022         3   9261          amarela
## 29 2023         3   9053          amarela
## 30 2024         3   9914          amarela
## 31 2015         4 458807          parda
## 32 2016         4 483026          parda
## 33 2017         4 497839          parda
## 34 2018         4 496106          parda
## 35 2019         4 509936          parda
## 36 2020         4 531808          parda
## 37 2021         4 587323          parda
## 38 2022         4 582469          parda

```

##	39	2023	4	561438	parda
##	40	2024	4	586563	parda
##	41	2015	5	3640	indígena
##	42	2016	5	3867	indígena
##	43	2017	5	3911	indígena
##	44	2018	5	4261	indígena
##	45	2019	5	4273	indígena
##	46	2020	5	5173	indígena
##	47	2021	5	5361	indígena
##	48	2022	5	5343	indígena
##	49	2023	5	5398	indígena
##	50	2024	5	5555	indígena
##	51	2015	9	0	ignorado
##	52	2016	9	5	ignorado
##	53	2017	9	0	ignorado
##	54	2018	9	0	ignorado
##	55	2019	9	0	ignorado
##	56	2020	9	0	ignorado
##	57	2021	9	0	ignorado
##	58	2022	9	0	ignorado
##	59	2023	9	0	ignorado
##	60	2024	9	0	ignorado
##	61	2015	NA	56919	0
##	62	2016	NA	52116	0
##	63	2017	NA	40646	0
##	64	2018	NA	37232	0
##	65	2019	NA	34453	0
##	66	2020	NA	34473	0
##	67	2021	NA	35974	0
##	68	2022	NA	28825	0
##	69	2023	NA	23084	0
##	70	2024	NA	20384	0

Remove NA in year

```
dados_appended_melted <- dados_appended_melted[ which(
  dados_appended_melted$year != "NA"), ]
```

Remove 0 in raça/cor

```
dados_appended_melted <- dados_appended_melted[ which(
  dados_appended_melted$raca_cor_factor != "0"), ]
```

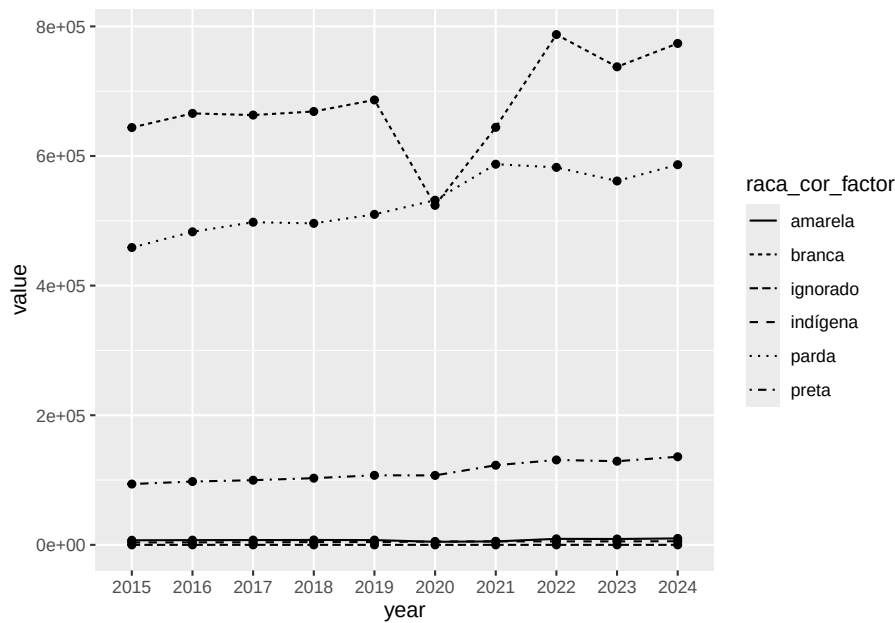
```
dados_appended_melted
```

##	year	variable	value	raca_cor_factor
## 1	2015	1	643928	branca
## 2	2016	1	665737	branca
## 3	2017	1	663147	branca
## 4	2018	1	668707	branca
## 5	2019	1	686474	branca
## 6	2020	1	523848	branca
## 7	2021	1	644210	branca
## 8	2022	1	787363	branca
## 9	2023	1	737590	branca
## 10	2024	1	773683	branca
## 11	2015	2	93830	preta
## 12	2016	2	97747	preta
## 13	2017	2	99745	preta
## 14	2018	2	102889	preta
## 15	2019	2	107306	preta
## 16	2020	2	107171	preta
## 17	2021	2	122852	preta
## 18	2022	2	131005	preta
## 19	2023	2	129047	preta
## 20	2024	2	135916	preta
## 21	2015	3	7051	amarela
## 22	2016	3	7276	amarela
## 23	2017	3	7375	amarela
## 24	2018	3	7524	amarela
## 25	2019	3	7359	amarela
## 26	2020	3	4716	amarela
## 27	2021	3	5313	amarela
## 28	2022	3	9261	amarela
## 29	2023	3	9053	amarela
## 30	2024	3	9914	amarela
## 31	2015	4	458807	parda
## 32	2016	4	483026	parda
## 33	2017	4	497839	parda
## 34	2018	4	496106	parda
## 35	2019	4	509936	parda
## 36	2020	4	531808	parda
## 37	2021	4	587323	parda
## 38	2022	4	582469	parda
## 39	2023	4	561438	parda

##	40	2024	4	586563	parda
##	41	2015	5	3640	indígena
##	42	2016	5	3867	indígena
##	43	2017	5	3911	indígena
##	44	2018	5	4261	indígena
##	45	2019	5	4273	indígena
##	46	2020	5	5173	indígena
##	47	2021	5	5361	indígena
##	48	2022	5	5343	indígena
##	49	2023	5	5398	indígena
##	50	2024	5	5555	indígena
##	51	2015	9	0	ignorado
##	52	2016	9	5	ignorado
##	53	2017	9	0	ignorado
##	54	2018	9	0	ignorado
##	55	2019	9	0	ignorado
##	56	2020	9	0	ignorado
##	57	2021	9	0	ignorado
##	58	2022	9	0	ignorado
##	59	2023	9	0	ignorado
##	60	2024	9	0	ignorado

Visualizar plot

```
ggplot(dados_appended_melted, aes(x=year, y=value, group=raca_cor_factor)) +
  geom_line(aes(linetype=raca_cor_factor)) +
  geom_point()
```



```
# wait to change the variable names:

# names(dados_appended_melted)[names(dados_appended_melted) == 'variable'] <- 'RACACOR'
# names(dados_appended_melted)[names(dados_appended_melted) == 'value'] <- 'n'
```

5.6 Mortalidade Anual Total por Neuroblastoma

```
dados_nb <- readRDS("./data/dados_nb.rds")

# Checar dados_nb
head(dados_nb)
```

##	CONTADOR	RACACOR	DTOBITO	CAUSABAS	DTNASC	IDADE	YMD	year
##	876...	17551	876	4	20012016	C749	10122010	405 2016-01-20 2016
##	5811...	56566	562030	1	14052018	C749	22042018	222 2018-05-14 2018

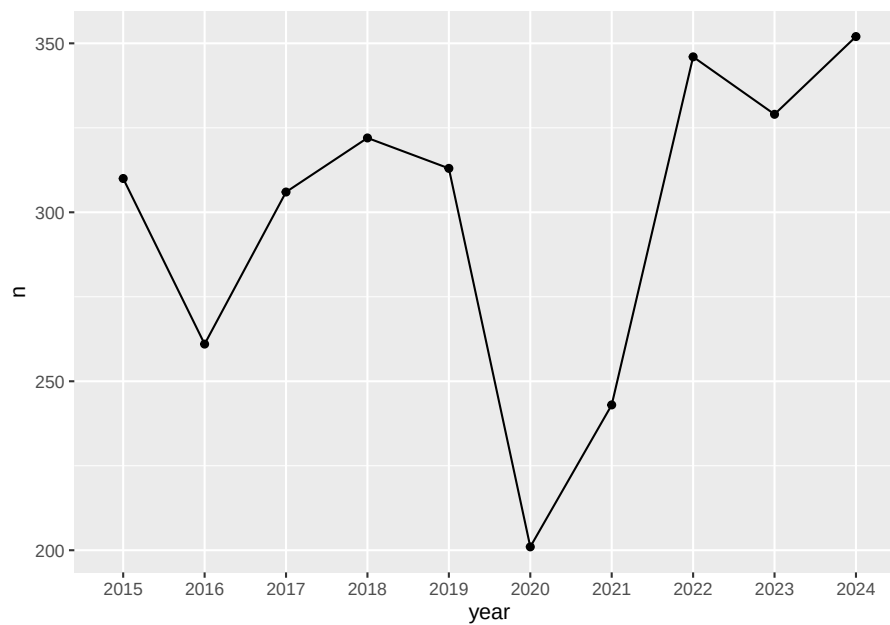
```
## 8831...77296      613331      1 11062019      C749 31011964      455 2019-06-11 2019
## 14106...82571     987213      4 01042019      C749 12021950      469 2019-04-01 2019
## 900...87692       78999      1 08032020      C749 02042014      405 2020-03-08 2020
## 8259...95051      470277      1 15022020      C749 21052005      414 2020-02-15 2020
```

```
dados_agregados_nb_year <- dados_nb %>%
  group_by(year) %>%
  summarise(n = n())

# Checar dados_agregados_nb_year
dados_agregados_nb_year
```

```
## # A tibble: 10 x 2
##   year      n
##   <chr> <int>
## 1 2015    310
## 2 2016    261
## 3 2017    306
## 4 2018    322
## 5 2019    313
## 6 2020    201
## 7 2021    243
## 8 2022    346
## 9 2023    329
## 10 2024    352
```

```
ggplot(data=dados_agregados_nb_year, aes(x=year, y=n, group=1)) +
  geom_line() +
  geom_point()
```

```
dados_nb_race_year_selcd <- dados_nb %>%
  select(RACACOR, year)
```

```
# Checar dados_nb_race_year_selcd
head(dados_nb_race_year_selcd)
```

```
##           RACACOR year
## 876...17551      4 2016
## 5811...56566     1 2018
## 8831...77296     1 2019
## 14106...82571    4 2019
## 900...87692      1 2020
## 8259...95051     1 2020
```

```
# dados_nb_race_2013_selcd <- dados_estados_nb_2013 %>%
#   select(RACACOR, CAUSABAS)
```

use `cast()` to convert data frame from long to wide format

```
library(reshape2)
```

```
##
## Attaching package: 'reshape2'
```

```
## The following object is masked from 'package:tidyr':
##
## smiths
```

```
## The following objects are masked from 'package:maditr':
##
##   dcast, melt
```

```
wide_dados_nb_appended <- dcast(dados_nb_race_year_selcd, year ~ RACACOR)
```

```
## Using year as value column: use value.var to override.
```

```
## Aggregation function missing: defaulting to length
```

Melt the data frame

```
dados_nb_appended_melted <- melt(wide_dados_nb_appended, id.vars = "year")
```

Obter variável raça/cor como palavras descritivas ao invés de números.

[illegible]

Remove NA in year

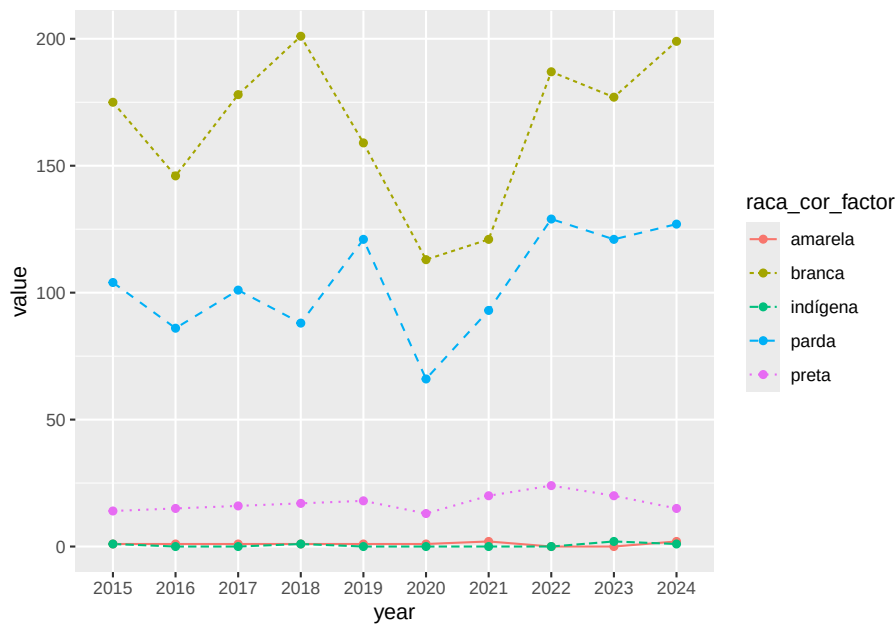
```
dados_nb_appended_melted <- dados_nb_appended_melted[ which(
  dados_nb_appended_melted$year != "NA"), ]
```

Remove 0 in raça/cor

```
dados_nb_appended_melted <- dados_nb_appended_melted[ which(
  dados_nb_appended_melted$raca_cor_factor != "0"), ]
```

Visualizar plot

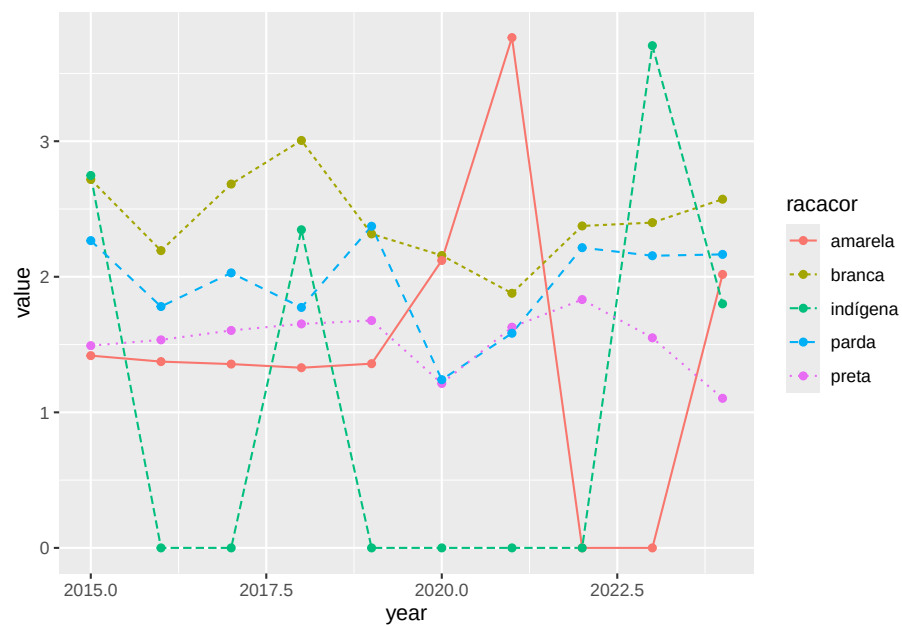
```
ggplot(dados_nb_appended_melted, aes(x=year, y=value, group=raca_cor_factor, color=raca_cor_factor)) +
  geom_line(aes(linetype=raca_cor_factor)) +
  geom_point()
```



Visualizar Plot Disparity

```
library('readxl')
Disparity_coefficient_Brazil_All <- read_excel("./data/Disparity_coefficient_Brazil_All.xlsx")

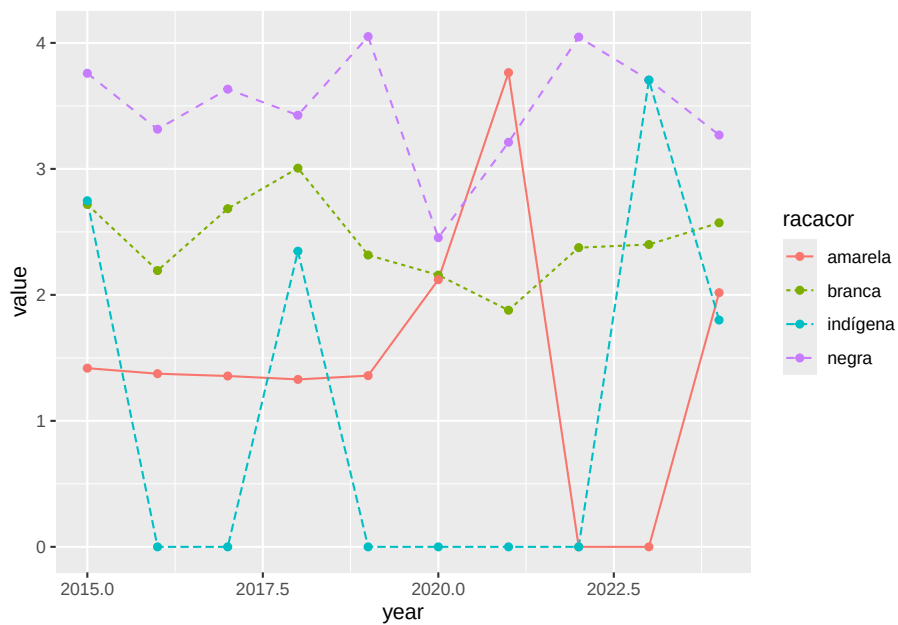
ggplot(Disparity_coefficient_Brazil_All, aes(x=year, y=value, group=racacor, color=racacor)) +
  geom_line(aes(linetype=racacor)) +
  geom_point()
```



Visualizar Plot Disparity

```
library('readxl')
Disparity_coefficient_Sum_PardosBlacks_Brazil_All <- read_excel("./data/Disparity_coefficient_Sum_PardosBlacks_Brazil_All.xlsx")

ggplot(Disparity_coefficient_Sum_PardosBlacks_Brazil_All, aes(x=year, y=value, group=racacor, color=racacor)) +
  geom_line(aes(linetype=racacor)) +
  geom_point()
```



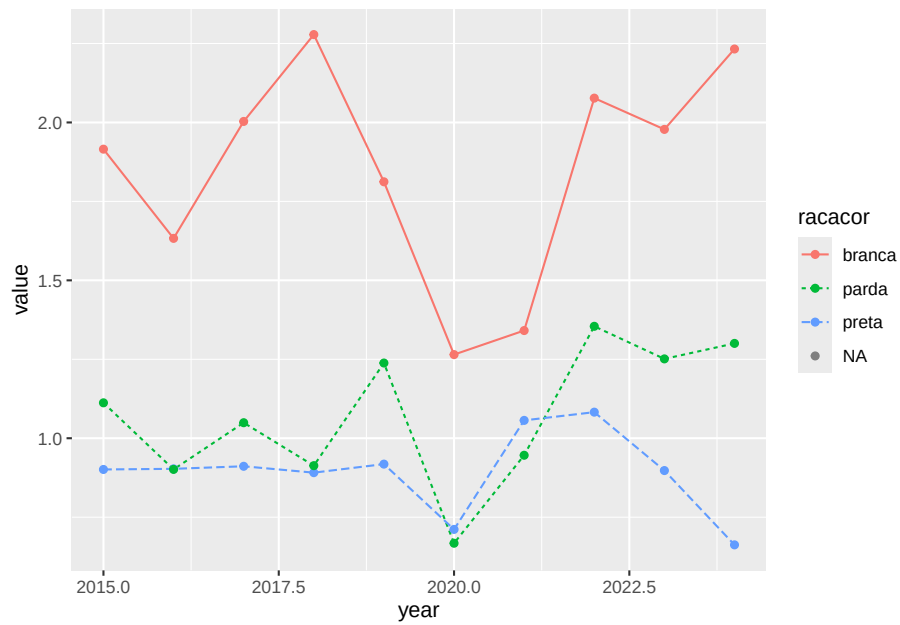
Visualizar Plot Disparity IBGE

```
library('readxl')
Disparity_ibge_plot <- read_excel("./data/Disparity_ibge_plot.xlsx")
```

```
ggplot(Disparity_ibge_plot, aes(x=year, y=value, group=racacor, color=racacor)) +
  geom_line(aes(linetype=racacor)) +
  geom_point()
```

```
## Warning: Removed 10 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

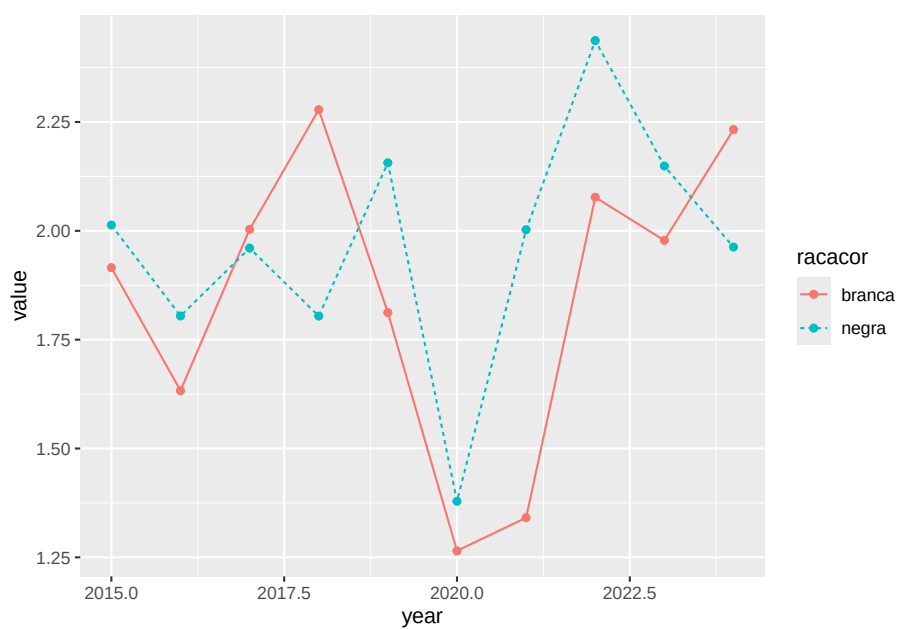
```
## Warning: Removed 10 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Visualizar Plot Disparity IBGE

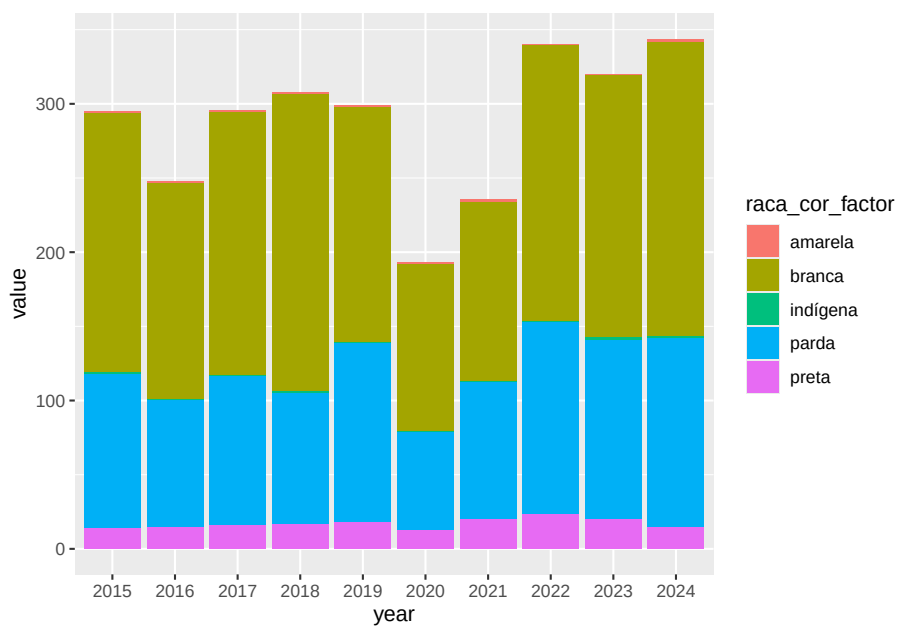
```
library('readxl')
Disparity_ibge_negros_plot <- read_excel("./data/Disparity_ibge_negros_plot.xlsx")
```

```
ggplot(Disparity_ibge_negros_plot, aes(x=year, y=value, group=racacor, color=racacor))
  geom_line(aes(linetype=racacor))+
  geom_point()
```



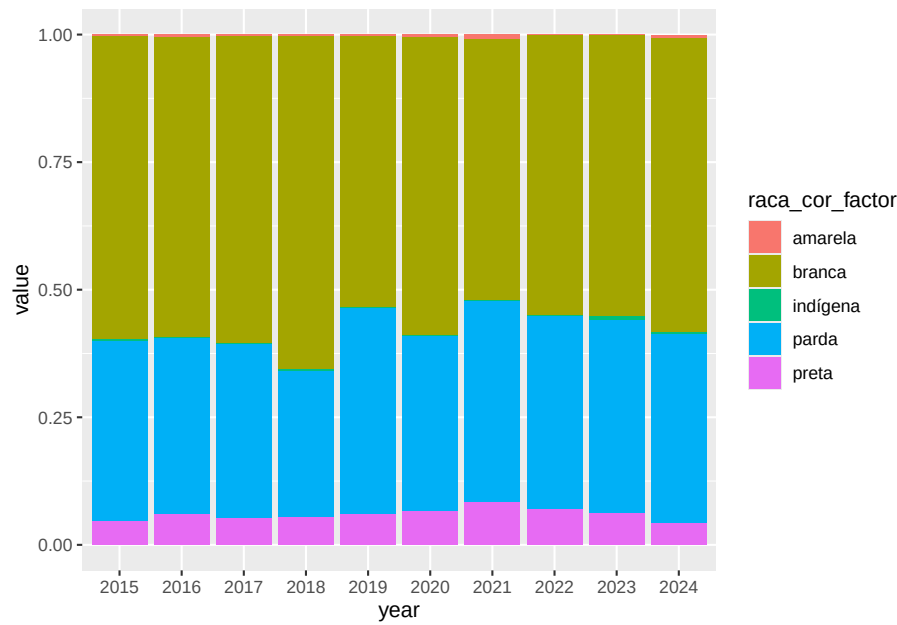
Stacked Graph

```
ggplot(dados_nb_appended_melted, aes(fill=raca_cor_factor, y=value, x=year)) +  
  geom_bar(position="stack", stat="identity")
```



Stacked + Graph

```
ggplot(dados_nb_appended_melted, aes(fill=raca_cor_factor, y=value, x=year)) +  
  geom_bar(position="fill", stat="identity")
```

Chapter 6

Final Words

We have finished a nice book.

Bibliography